Utilisation de l'intelligence artificielle dans le projet **Promptly**

Rapport : outils d'IA (developpement) et IA integree au produit

Encadre par : **Khemiri Akram**

Équipe Alpha Code

Lamis Ben Hassine
Khalil Fray
Saif Denden
Isra Bel Habib
Hamza Taychi

Annee universitaire 2025–2026

16 octobre 2026

Résumé

Ce rapport documente l'utilisation de l'intelligence artificielle dans le projet **Promptly**, une plateforme d'apprentissage en ligne (Spring Boot + Angular). Deux axes sont distingues :

1. l'**IA comme outil de developpement** (ChatGPT, puis editeur Cursor avec modeles Claude, Grok, etc.), avec prompts, resultats, corrections et captures d'ecran ;
2. l'**IA integree a l'application** (Ollama, APIs externes, microservices ML), avec justification, fonctionnement et resultats observes.

Table des matières

1 Introduction **3**
- 1.1 Objectif du rapport . . . 3
- 1.2 Deux faces de l'IA . . . 3
- 1.3 Methode de documentation . . . 3
- 1.4 Organisation du document . . . 3

2 Role du projet Promptly **4**
- 2.1 Contexte . . . 4
- 2.2 Fonctionnalites principales . . . 4
- 2.3 Pile technique . . . 4
- 2.4 Roles utilisateurs . . . 4

3 IA utilisee pour developper le projet **5**
- 3.1 Outils et modeles . . . 5
- 3.2 Chaine initiale : ChatGPT puis Cursor . . . 5
  - 3.2.1 Prompt de l'auteur a ChatGPT . . . 5
  - 3.2.2 Resultat ChatGPT : prompt pour Cursor . . . 6
- 3.3 Sessions Cursor par gestion . . . 8
  - 3.3.1 S0 — Fondation (template Spring Boot + Angular) . . . 9
  - 3.3.2 S1 — Authentification et utilisateurs . . . 10
  - 3.3.3 S2 — Cours et chapitres . . . 12
  - 3.3.4 S3 — Evaluations / Quiz . . . 15
  - 3.3.5 S4 — Forum . . . 20
  - 3.3.6 S5 — Reclamations . . . 23
  - 3.3.7 S6 — Layout (navbar / sidebar) . . . 27
  - 3.3.8 S7 — IA pedagogique (Ollama : resume, flashcards) . . . 29
  - 3.3.9 S8 — Assistant IA (chatbot) . . . 31
  - 3.3.10 S9 — Recrutement + matching CV . . . 32
- 3.4 Synthese des sessions . . . 35

4 IA integree a l'application **37**
- 4.1 Vue d'ensemble . . . 37
- 4.2 Ollama (LLM local) . . . 37
  - 4.2.1 Usage . . . 37
  - 4.2.2 Pourquoi . . . 37
  - 4.2.3 Resultat (captures) . . . 38
- 4.3 APIs externes . . . 38
  - 4.3.1 Hugging Face — emotion . . . 38
  - 4.3.2 Sightengine — moderation . . . 38
  - 4.3.3 TextGears — auto-correction . . . 38
  - 4.3.4 Google Gemini — matching CV . . . 38

4.4 Microservices ML locaux 39

5 Conclusion **40**
5.1 Bilan 40
5.2 Limites 40
5.3 Perspectives 40

Chapitre 1

Introduction

1.1 Objectif du rapport

Ce rapport documente l'utilisation de l'intelligence artificielle dans le projet **Promptly**. Il presente :

— les **outils d'IA** utilises pour concevoir et coder l'application ;

— les **fonctionnalites d'IA** integrees au produit livre.

1.2 Deux faces de l'IA

1. **IA cote developpement.** ChatGPT a servi a preparer le premier grand prompt. L'editeur **Cursor** (mode Agent) a ensuite heberge le dialogue avec des **modeles** tels que **Claude** et **Grok**, qui ont genere, corrige et explique le code. Chaque etape importante est documentee : prompt, resultat, eventuel probleme, prompt de correction, nouveau resultat — avec captures d'ecran.
2. **IA cote application.** Le produit appelle des modeles et des APIs (Ollama, Hugging Face, Sightengine, TextGears, Gemini, services ML locaux) pour resume, chat, moderation, emotion, matching CV, etc.

1.3 Methode de documentation

Pour chaque echange significatif avec une IA de developpement :

1. capture du **prompt** ;
2. capture du **resultat** ;
3. si erreur ou ecart : capture du **probleme**, du **prompt de correction**, puis du **resultat corrige**.

Les images sont placees dans `figures/chatgpt/`, `figures/cursor/` et `figures/app/`.

1.4 Organisation du document

Le chapitre 2 rappelle brievement le role de Promptly. Le chapitre 3 detaille la chaine ChatGPT → Cursor (Claude / Grok). Le chapitre 4 presente l'IA integree au produit. Le chapitre 5 conclut.

Chapitre 2

Role du projet Promptly

2.1 Contexte

Promptly est une plateforme web d'apprentissage dediee a la programmation (Java, Spring Boot, Angular, algorithmique). Elle s'adresse aux etudiants, aux enseignants et a un administrateur.

2.2 Fonctionnalites principales

— **Cours et chapitres** : contenus, PDF, resume IA, flashcards ;
— **Quiz** : QCM, scores, detection GUESS/KNOW (ML) ;
— **Forum** : discussions, auto-correction (TextGears), censure (Sightengine) ;
— **Reclamations** : depot etudiant, labels ML + emotion (admin) ;
— **Recrutement** : offres (admin), candidature + CV, matching competences (Gemini) ;
— **Assistant IA** : chatbot general (Ollama).

2.3 Pile technique

Couche	Technologie
Frontend	Angular
Backend	Spring Boot 3
Base de donnees	MySQL
LLM local	Ollama
ML locaux	Python (FastAPI) — ports 8090 / 8091

TABLE 2.1 – Pile technique de Promptly

2.4 Roles utilisateurs

— **ETUDIANT** : cours, quiz, forum, reclamations, candidatures, assistant ;
— **ENSEIGNANT** : gestion des cours / quiz (+ meme acces apprenant) ;
— **ADMIN** : utilisateurs, traitement reclamations, offres de recrutement, vues ML.

Chapitre 3

IA utilisee pour developper le projet

3.1 Outils et modeles

Il faut distinguer l'**environnement** (ou l'on ecrit les prompts) et le **modele d'IA** (qui produit les reponses).

Element	Role
ChatGPT (web)	Premier echange : expression du besoin ; generation du prompt structure a coller ensuite dans Cursor.
Cursor	Editeur / IDE (mode Agent) : interface de chat, fichiers du projet, terminal. Ce n'est pas le modele lui-meme.
Claude (Anthropic)	Modele utilise dans Cursor pour generer et corriger le code (ex. Claude 5.5 / Sonnet).
Grok (xAI)	Autre modele propose dans Cursor, egalement utilise selon les besoins.

TABLE 3.1 – Outils (environnement) et modeles d'IA de developpement

En resume : **Cursor** = **editeur** ; **Claude** / **Grok** = **IA** qui repondent aux prompts a l'interieur de Cursor. ChatGPT a ete utilise en amont pour preparer le premier prompt. Les sessions ci-dessous sont reconstituees a partir de l'**historique de chat** Cursor (prompts reels + resultats observes).

3.2 Chaine initiale : ChatGPT puis Cursor

3.2.1 Prompt de l'auteur a ChatGPT

Prompt utilisateur → ChatGPT

i will make acadamique project so the project about elearning named promptly for giving courses about coding like springboot java angular ... and algorithme the gestion of the project is reclamation, cours and chapter, evaluation like small test quiz, forome like samll comunite all the student will make postes and told to each other so this project about how to use the ia to make the project like i use currsor or chatgpt the prffesor told as to give he screen shot of the promte and the result after the prompt and correction of the promte if we need it and also how we use the ia in the project like how we use olama if we work with it and how we use api extern in the project exetera bacouse we will make rapport about this in the end so give me promte give it to the currsor to make this and dont forget the

content of the rapport becouse if we need screenshot lets make it to put in in the rapport like i tolde u the part how we use ia in the porcet and how we use ia to make the project also told him to begin with the template then we need to make gestion by gestion don't make the full project in one prmote the project with springboot and angular lets begin with the template then make the other gestion step by step each step need confiramtion

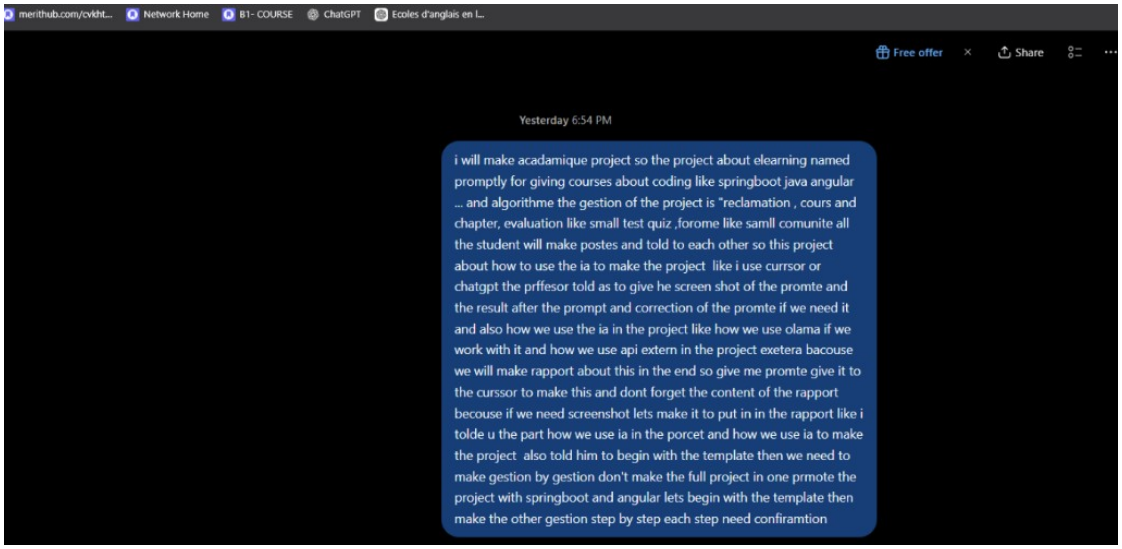


FIGURE 3.1 – Capture — premier prompt envoye a ChatGPT

3.2.2 Resultat ChatGPT : prompt pour Cursor

Reponse ChatGPT — resume du prompt pour Cursor

Stack. Backend : Java 17, Spring Boot, Web, JPA, Security, Validation, Maven, MySQL. Frontend : Angular, TypeScript, Router, services, Reactive Forms, UI responsive.
Architecture. Monolithe : un backend Spring Boot, un frontend Angular, une base MySQL. Packages classiques ; pas de microservices.
Contraintes. Interdit : Eureka, API Gateway, Docker/Kubernetes. Execution locale ; Spring Boot connecte directement a MySQL.
IA (plus tard). Ollama et APIs externes seulement apres validation.
Workflow etape 1 (template). Initialiser Spring Boot + Angular, MySQL, page d'accueil, health-check ; attendre confirmation avant chaque module suivant.

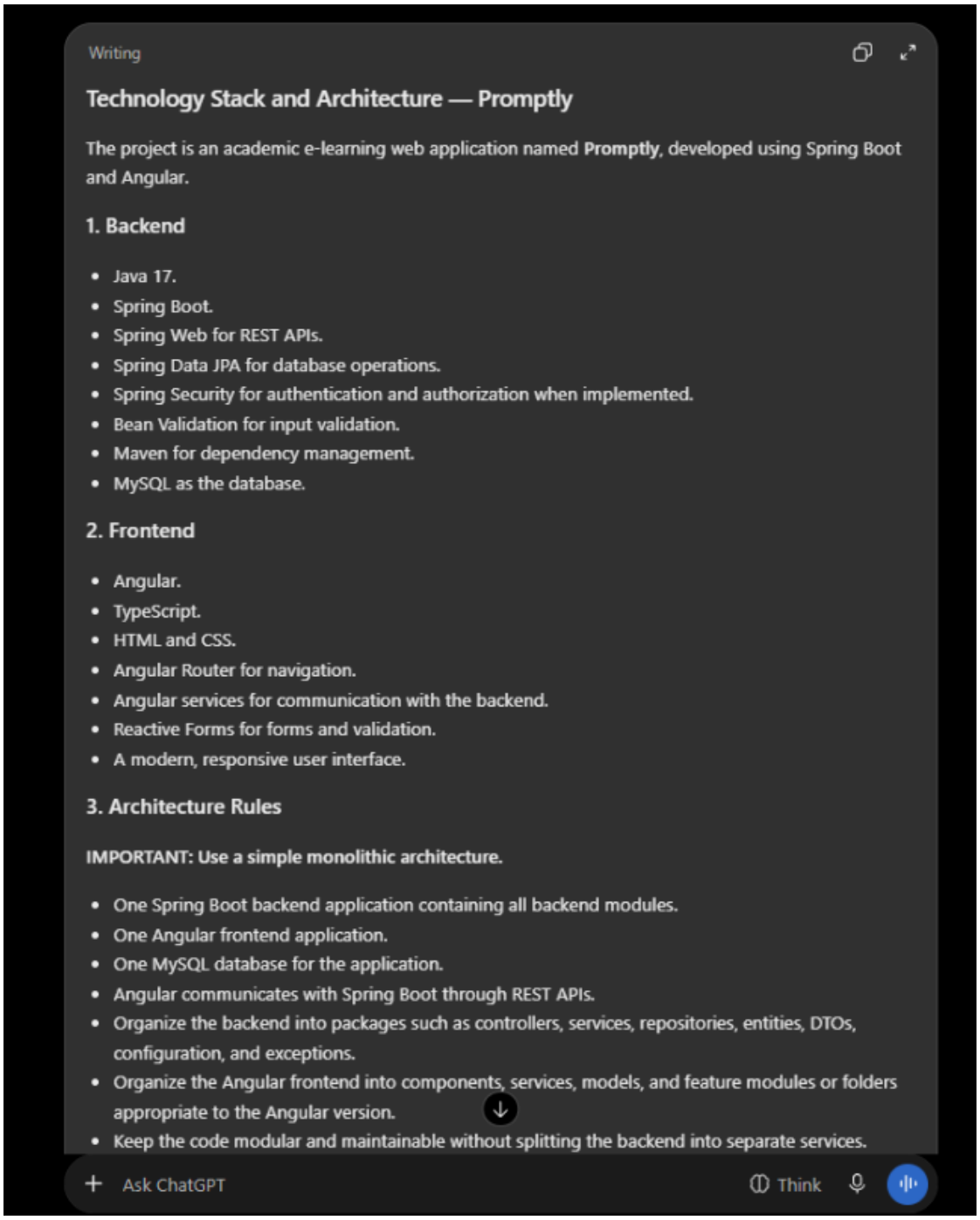


FIGURE 3.2 – Capture — prompt Cursor genere par ChatGPT (stack et architecture)

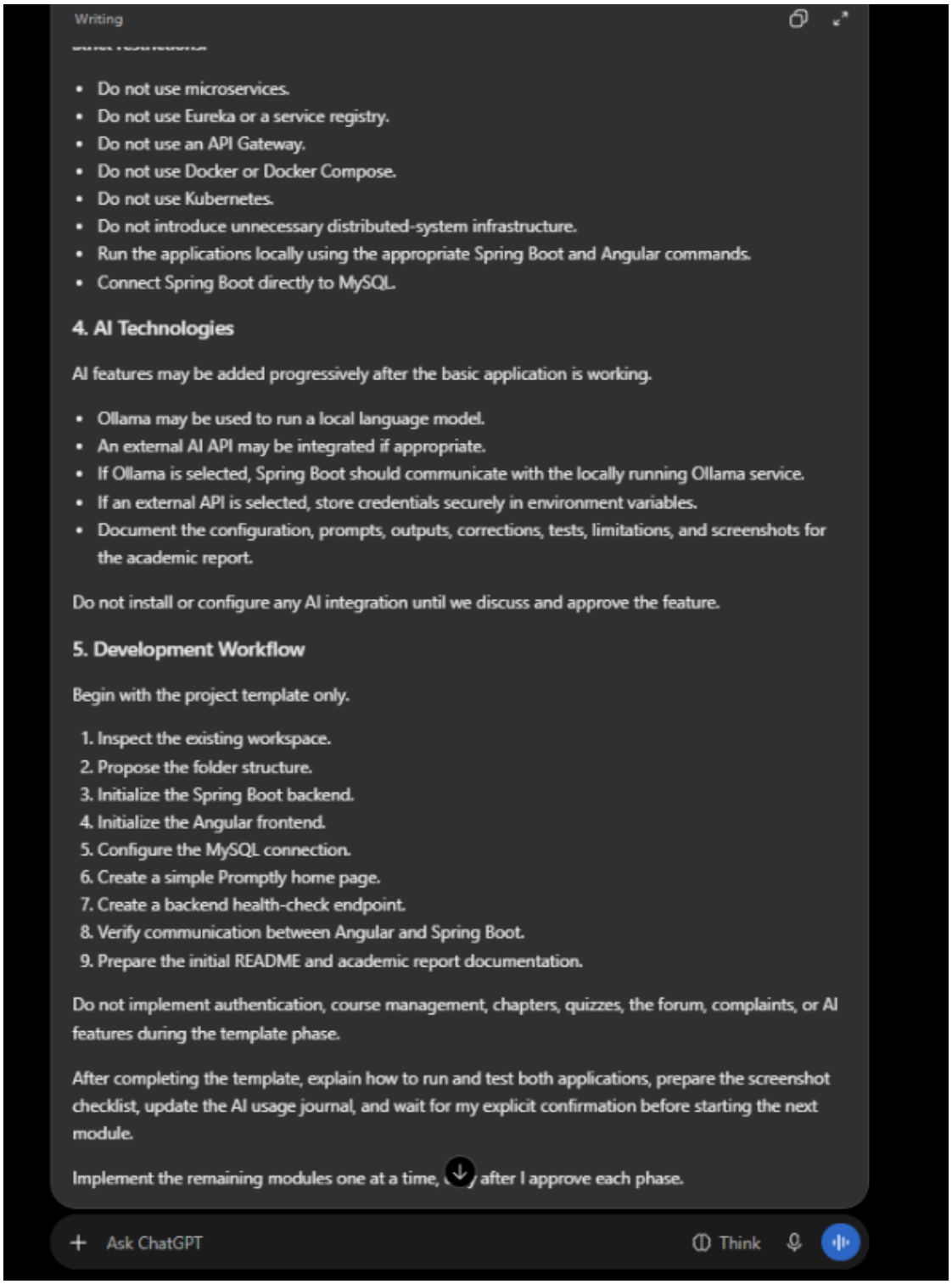


FIGURE 3.3 – Capture — suite du prompt Cursor (contraintes, IA, workflow template)

3.3 Sessions Cursor par gestion

Chaque sous-section = une **gestion** (ou un lot de prompts lies). Format : **Prompt** → **Resultat** (et corrections si un probleme a ete corrige dans le chat). Modele dans Cursor : **Claude 5.5** / **Grok** (selon la session).

3.3.1 S0 — Fondation (template Spring Boot + Angular)

Modele : Claude / Grok (Cursor Agent)

Prompt (issu de ChatGPT, colle dans Cursor)

Academic E-Learning Project — Promptly. Develop progressively, management by management, confirm each step. Phase 0 / template only : inspect workspace, propose folder structure, initialize Spring Boot backend, Angular frontend, configure MySQL, simple Promptly home page, health-check endpoint, verify Angular ↔ Spring Boot. Do not implement auth, courses, quizzes or AI yet.

Resultat

Workspace initialise : backend Spring Boot (Maven, MySQL, `/api/health`), frontend Angular, README. Communication Angular ↔ API verifiee. Attente de confirmation avant le module suivant.



FIGURE 3.4 – Capture Promptly — health-check API (`/api/health`)

3.3.2 S1 — Authentification et utilisateurs

Prompt

Implement Module 1 — Authentication and User Management : registration, login, JWT, roles (ETUDIANT / ENSEIGNANT / ADMIN), admin user list, access restrictions.

Resultat

Auth complete : inscription, login JWT, roles, page admin utilisateurs. Tests backend `BUILD SUCCESS` ; routes protegees (`401`/`403` verifies). Compte admin initialise via variables d'environnement.

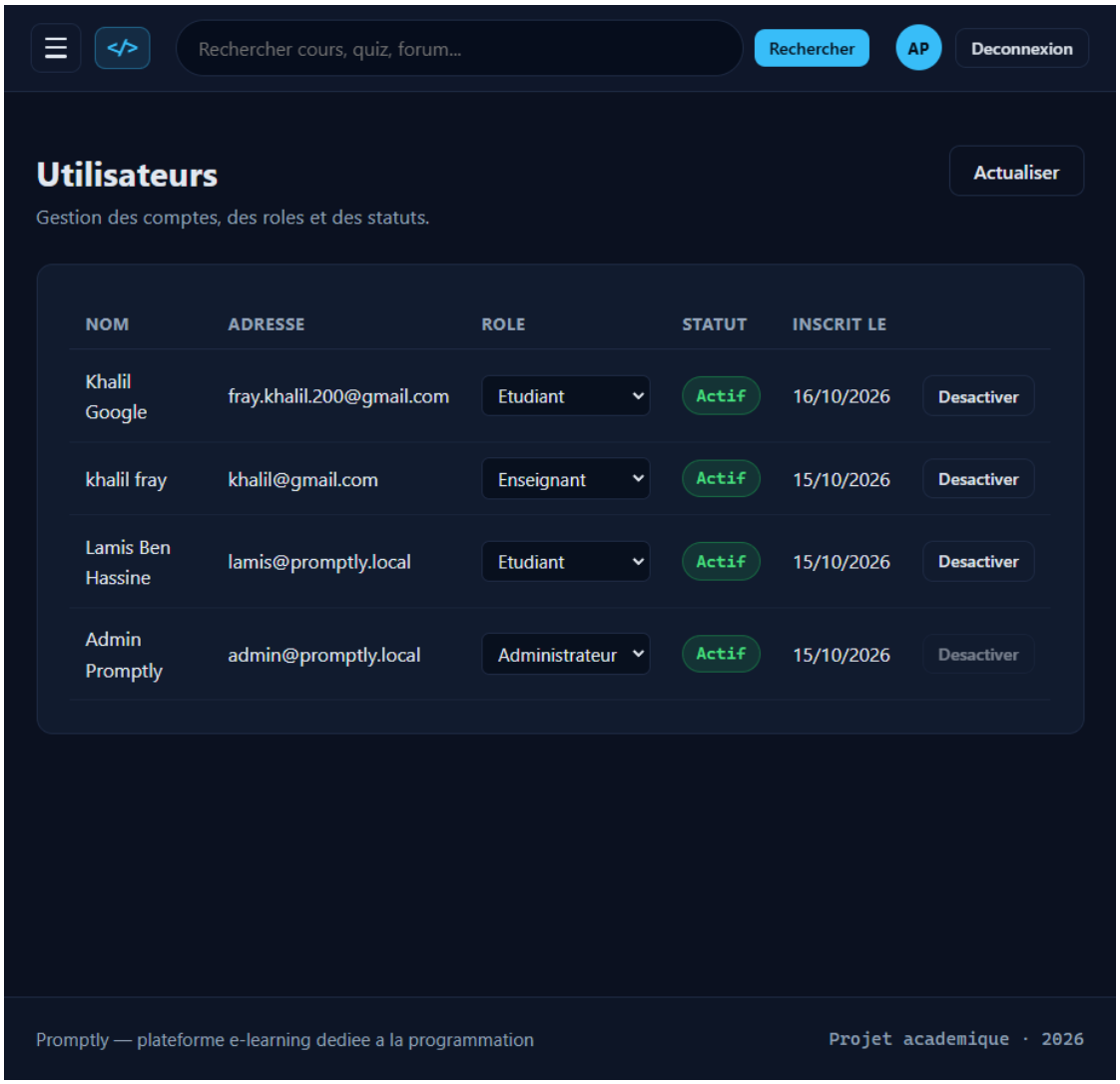


FIGURE 3.5 – Capture Promptly — administration des utilisateurs

Prompt suivant — Google Sign-In

lets make in the user part login with gmail make the code and tell me what i have to add like i have to take token ...

Resultat

Bouton Google sur Connexion / Inscription. Frontend envoie l'ID token a `POST /api/auth/google` ; backend verifie avec le Client ID puis emet le JWT Promptly. Configuration : `googleClientId` (frontend) + `PROMPTLY_GOOGLE_CLIENT_ID`.

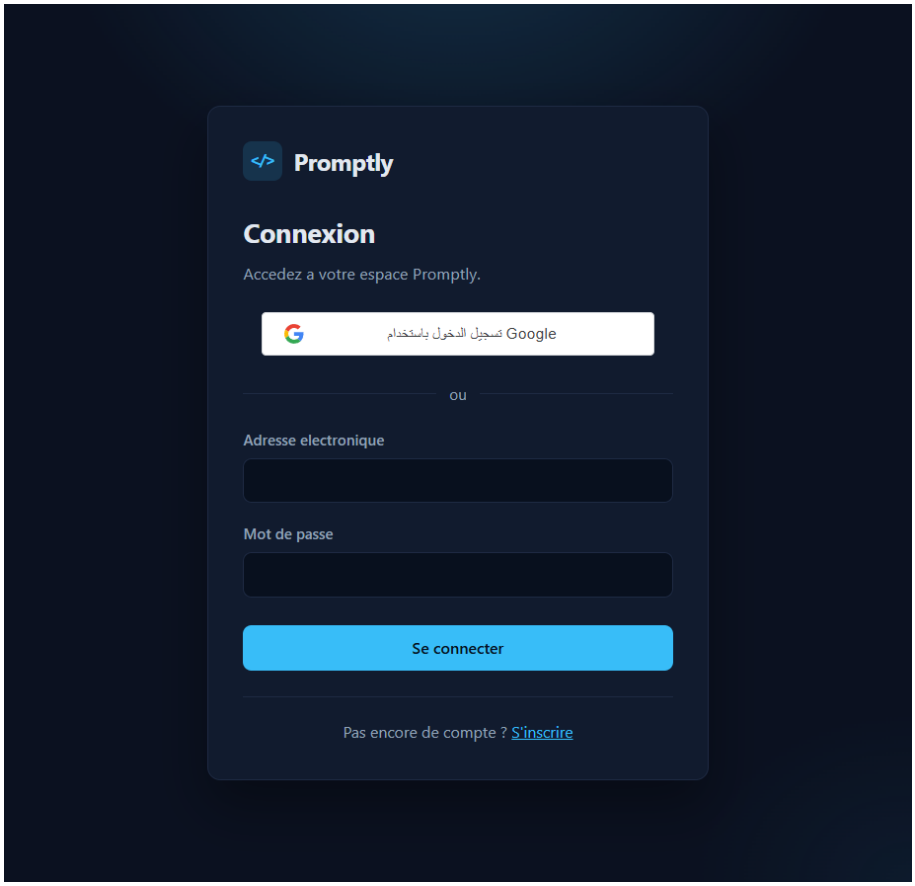


FIGURE 3.6 – Capture Promptly — Connexion avec bouton Google

Prompt — page d'accueil

the first page make it like login or gigin up olny without the nav and side bar

Resultat

/ redirige vers Connexion (ou Cours si deja connecte). Pages Connexion / Inscription sans navbar ni sidebar. Bouton Google visible sur la page de connexion.

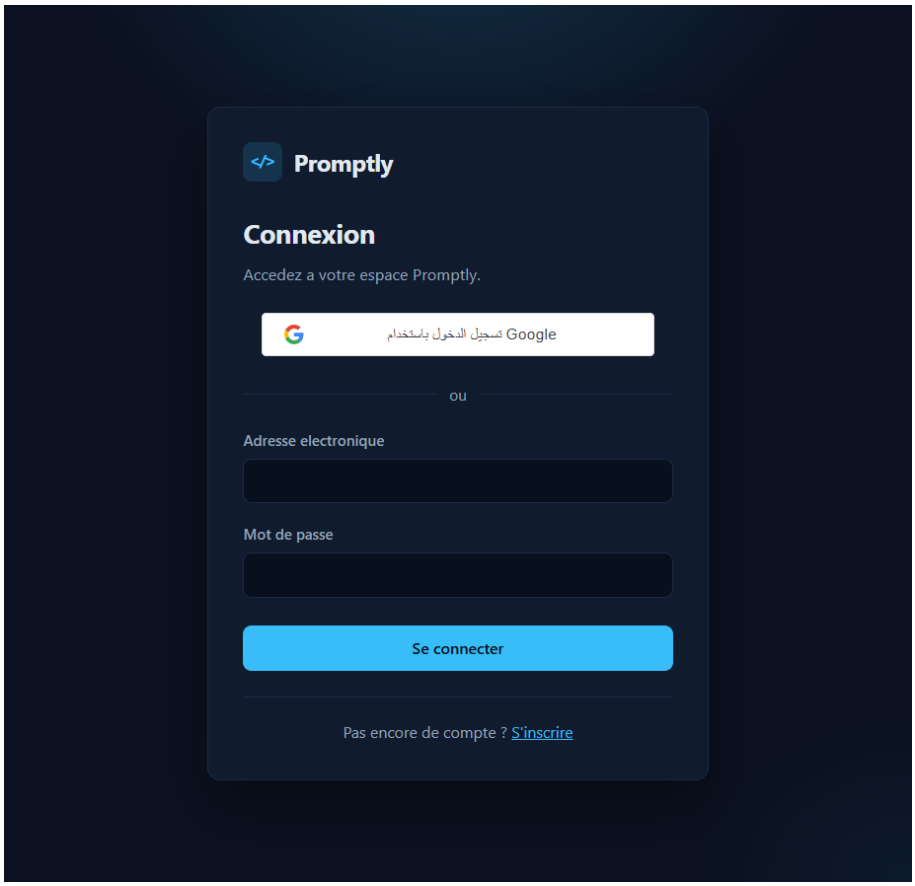


FIGURE 3.7 – Capture Promptly — page Connexion (sans nav/sidebar) + Google

3.3.3 S2 — Cours et chapitres

Prompt

implement module 2

Resultat

CRUD cours : categories (Java, Spring Boot, Angular, ...), publication, roles (etudiant = publies ; enseignant = ses cours ; admin = tous). Routes `/cours`, formulaires, catalogue filtre.

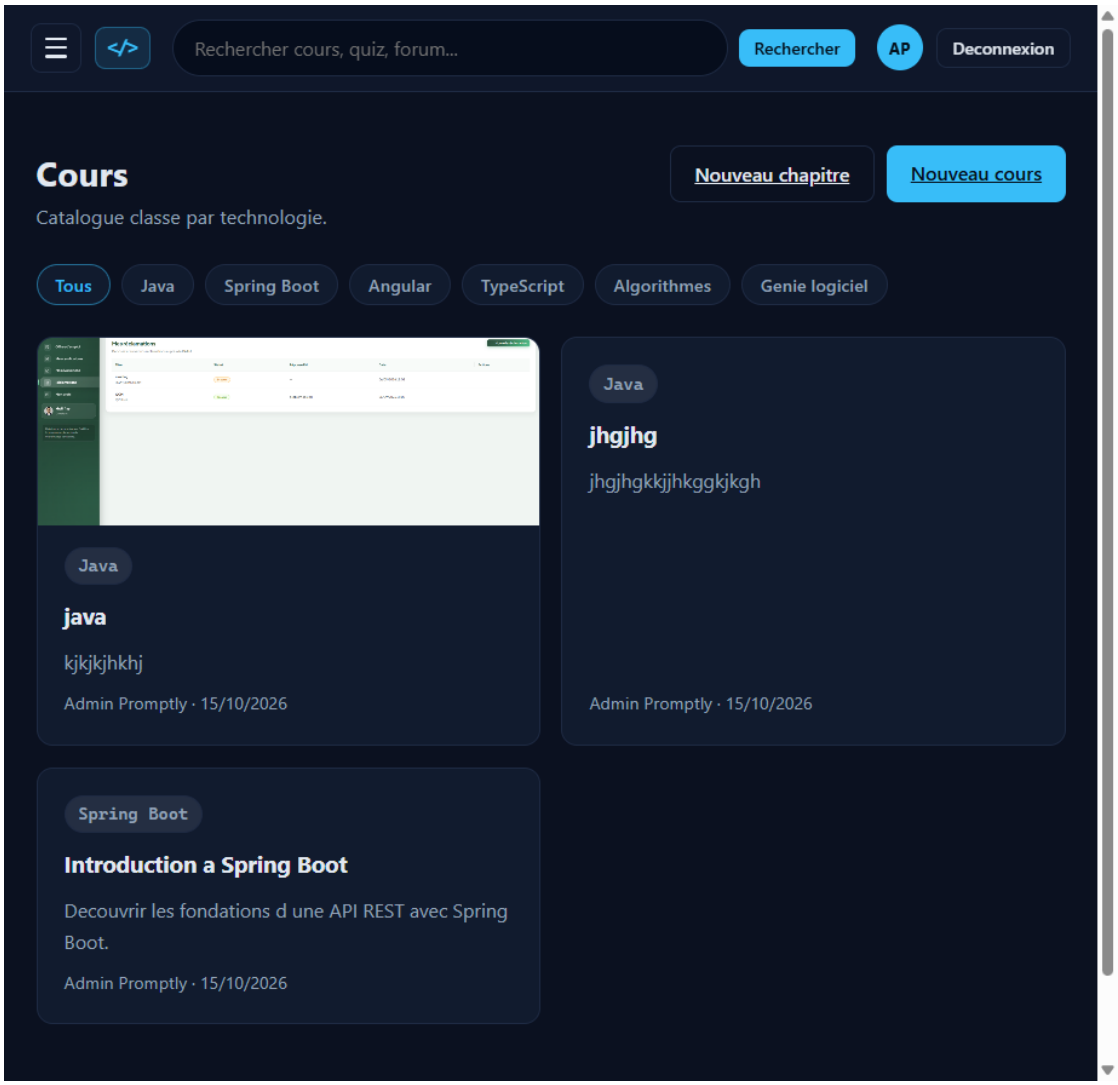


FIGURE 3.8 – Capture Promptly — catalogue des cours

Prompt

add img to the cours and upload pdf to the cours and chapter to the cours

Resultat

Upload image de couverture + PDF cours ; chapitres lies au cours avec PDF ; fichiers servis via `/api/files/**` (dossier `uploads/`).

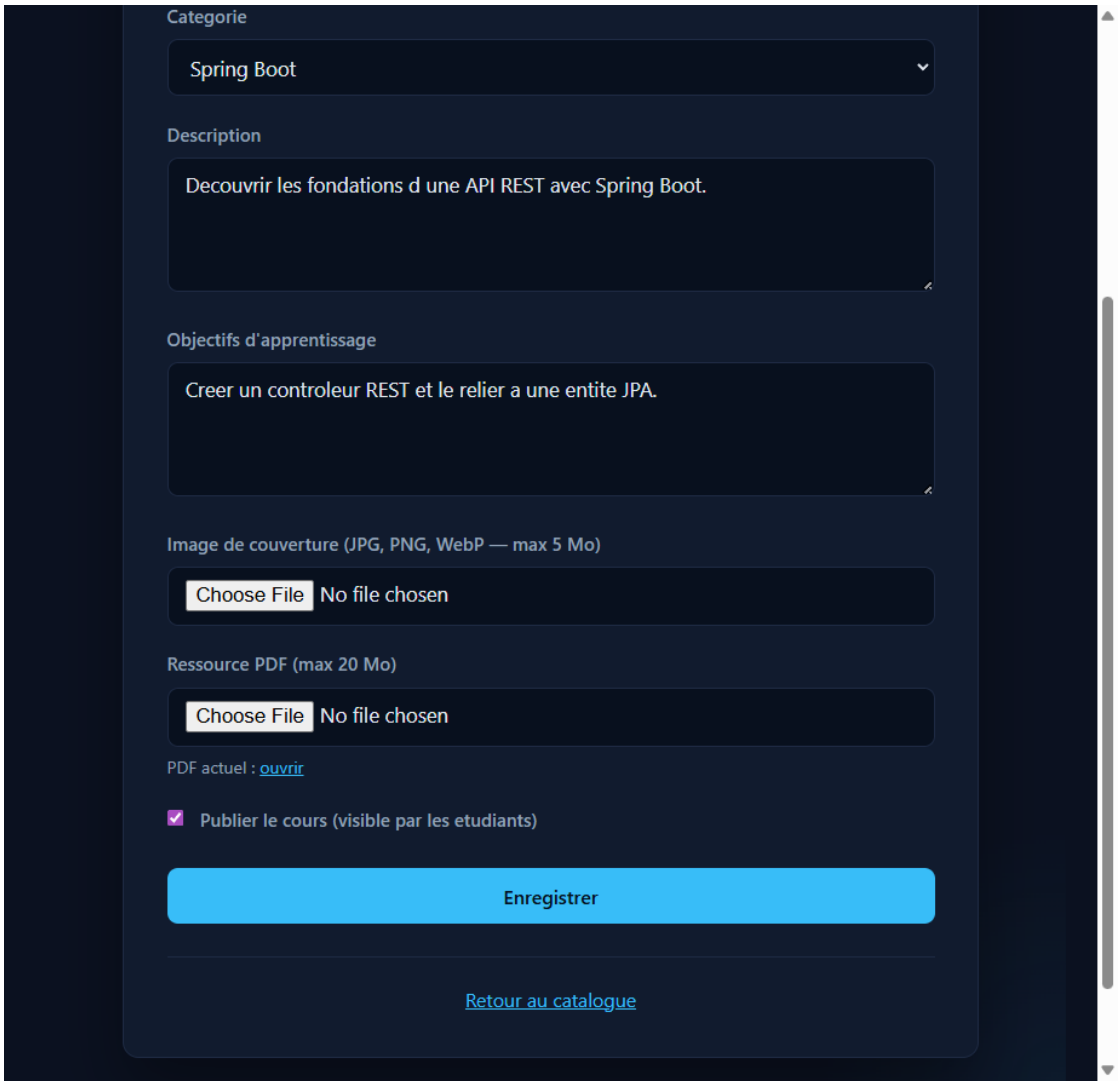


FIGURE 3.9 – Capture Promptly — formulaire cours (image de couverture + PDF)

Prompt

make it when i add chapter i choose the cours of the chapter

Resultat

Formulaire « Nouveau chapitre » avec liste deroulante de cours (presaisie si on vient d'une fiche cours).

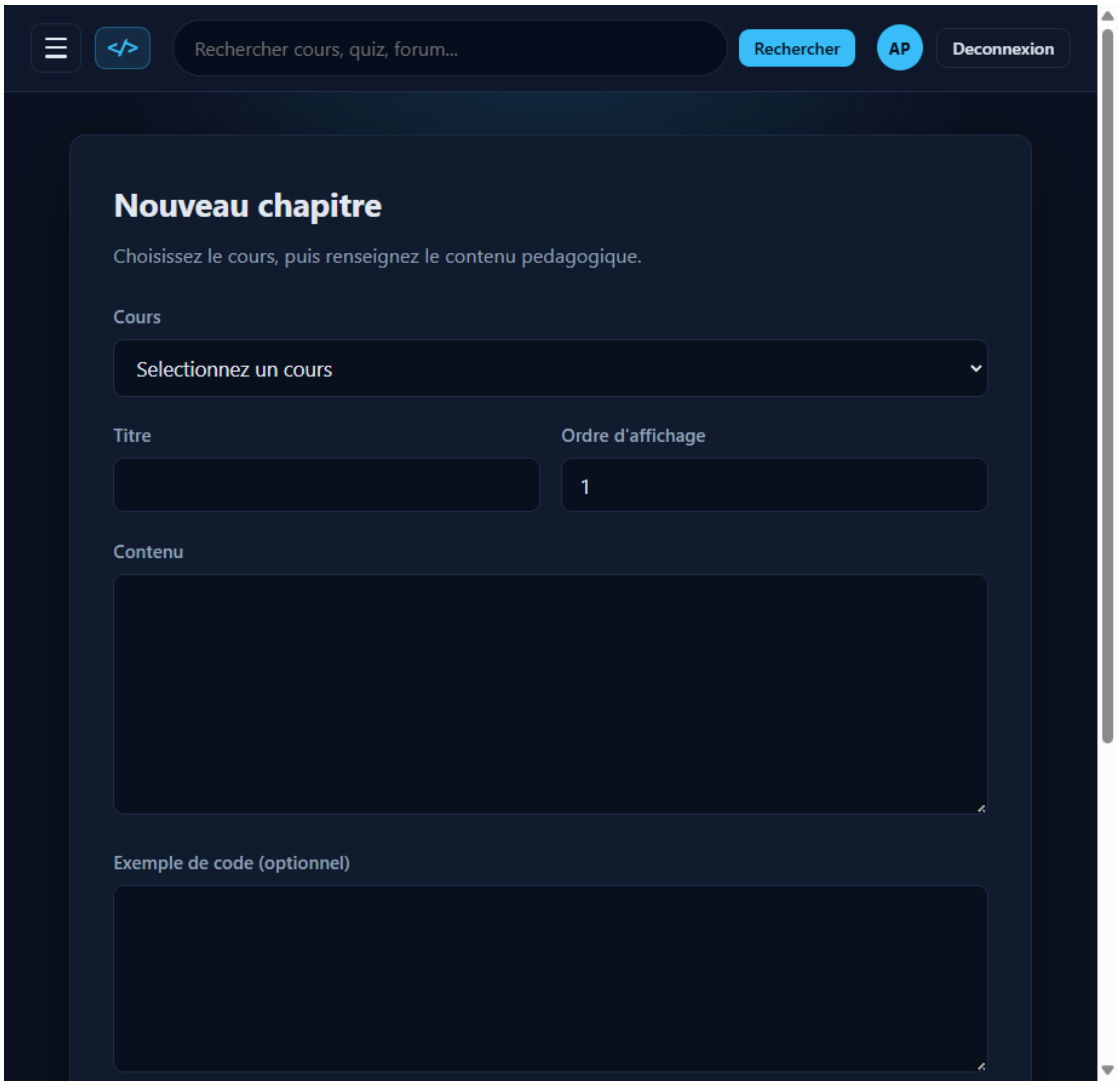


FIGURE 3.10 – Capture Promptly — Nouveau chapitre (choix du cours)

3.3.4 S3 — Evaluations / Quiz

Prompt

MAKE THE NEXT GESTION

Resultat

Module Quiz : QCM lies a un cours (chapitre optionnel), passage etudiant, score ; enseignants voient les tentatives. Nav `/quiz`.

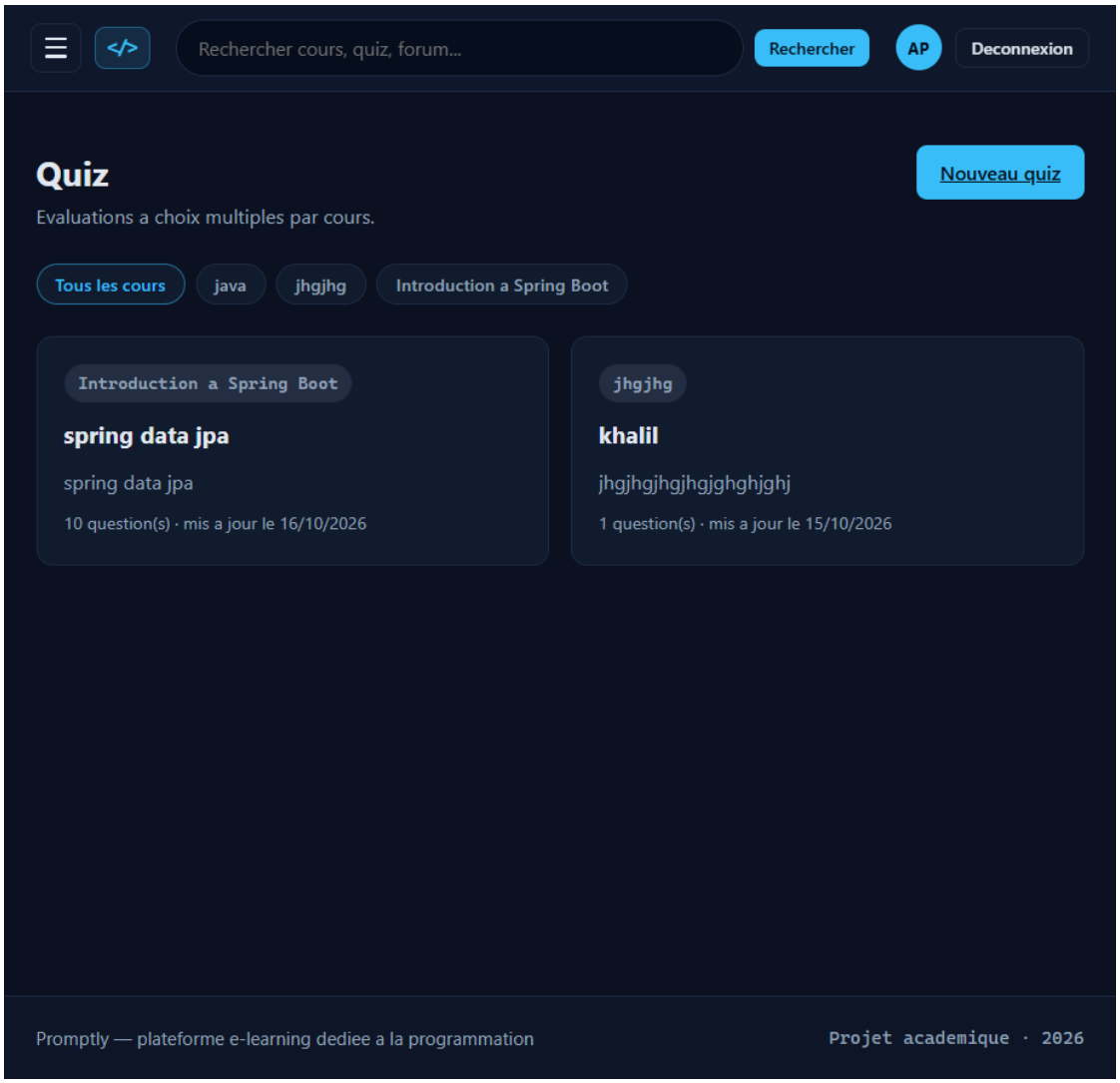


FIGURE 3.11 – Capture Promptly — liste des quiz

Prompt

now i need to make in the evaluation generation quiez with ollma when i add the titre and description and when i slect the cours of the quiz take all this then tell the ollma to make quiz with 10 quetions

Resultat

Bouton « Generer 10 questions avec IA » : Ollama utilise titre, description et contenu du cours/chapitre/PDF pour proposer 10 QCM a revoir avant creation.

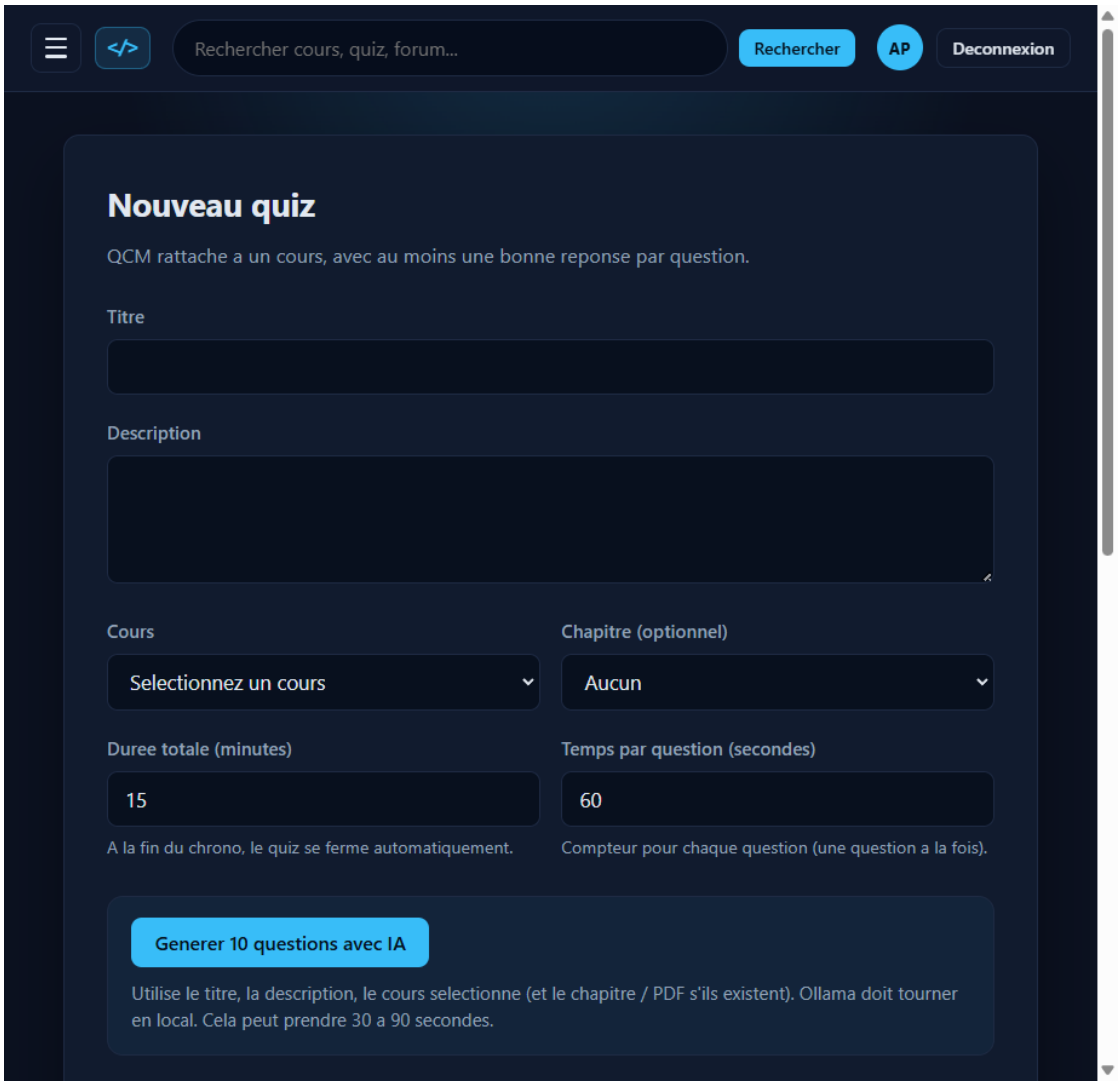


FIGURE 3.12 – Capture Promptly — Nouveau quiz (timers + Generer 10 questions avec IA)

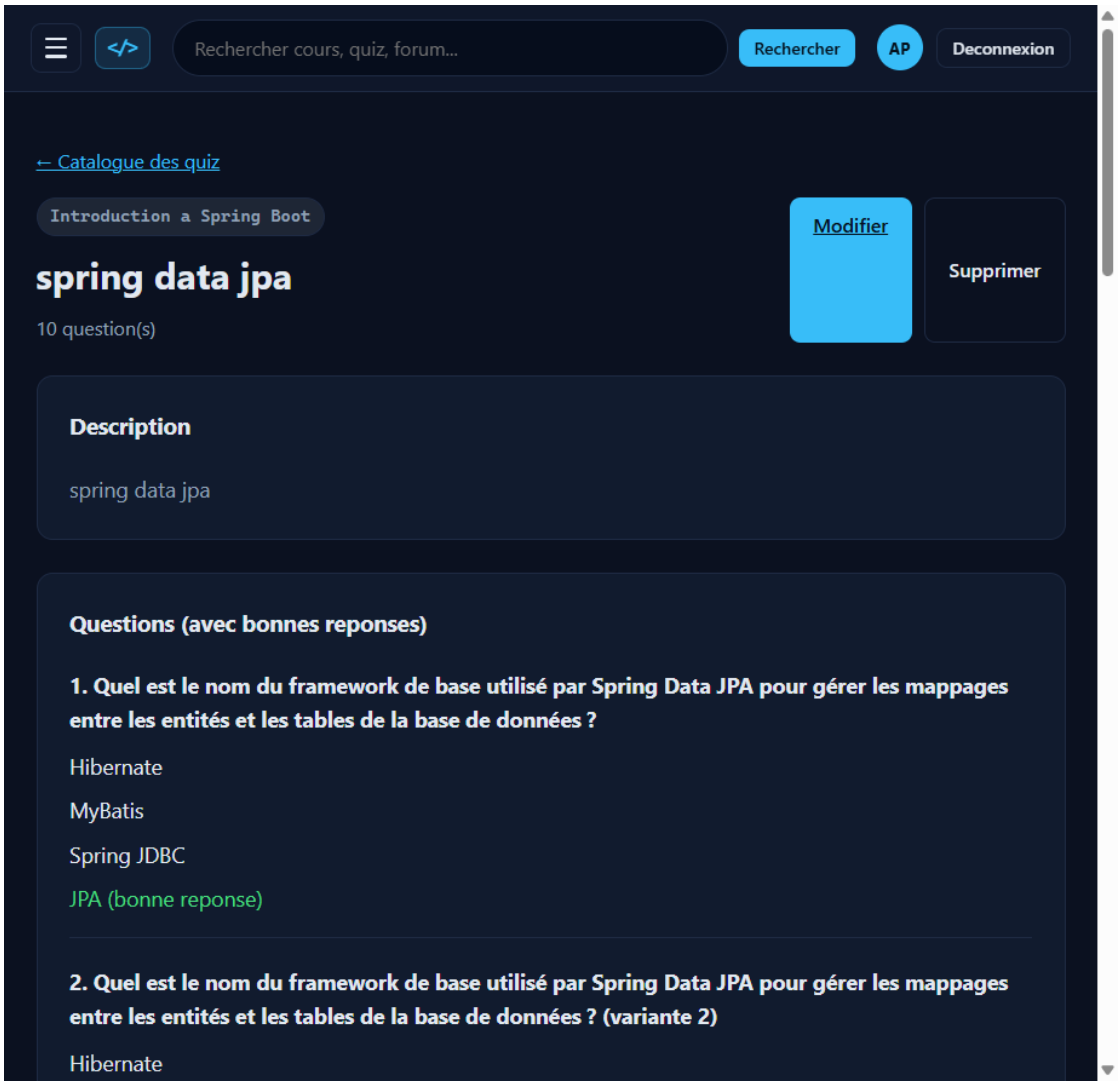


FIGURE 3.13 – Capture Promptly — detail d'un quiz genere (questions + bonnes reponses)

Prompt

make the student see the score nad the good reponse and the bad one and make the techer can see all the student make the quiz with the result and maken him can edit the result

Resultat

Etudiant : score + correction (bonnes/mauvaises reponses). Enseignant : liste des tentatives, detail, modification de note.

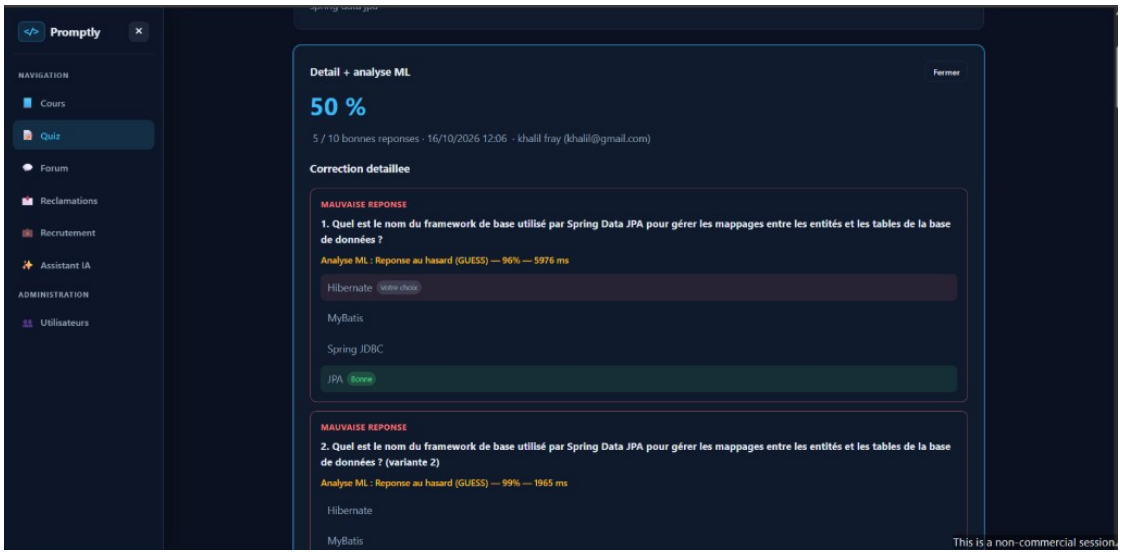


FIGURE 3.14 – Capture Promptly — score et correction detaillee d'une tentative

Prompt

make the quiz like this when i make the quiz i can add time when the time is over the quiz will close and the quetion will come one by one not all the quetion in the same page and make counter olso to each quetion then make programe with machine learing then connect it to the prject the programe predect the student when he check the answer he check it randomly or he know the answer make onlly the techer see the guess or know the answer

Resultat

Minuteur global + par question ; une question a la fois ; auto-soumission. Service Python `ml-integrity` (port 8090, RandomForest) predit `GUESS`/`KNOW` — visible uniquement par l'enseignant.

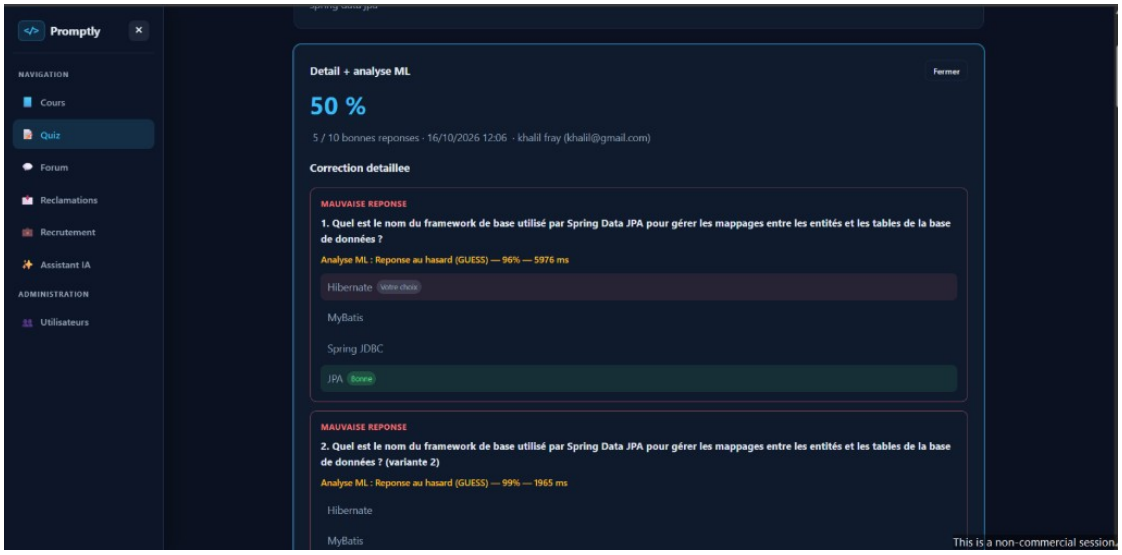


FIGURE 3.15 – Capture Promptly — Detail + analyse ML (labels GUESS / KNOW visibles uniquement pour l'enseignant)

Probleme puis correction

Prompt probleme : when i click detail +ml no thing is happen
Correction : le panneau se mettait a jour hors ecran ; scroll + section ML plus visible.
Autre : make modification when i make wrong reponse don't tell me he know the reponse → reponse fausse forcee en `GUESS`.

3.3.5 S4 — Forum

Prompt

next gestion

Resultat

Forum : sujets par categorie, commentaires, moderation / verrouillage enseignants et admins. Nav `/forum`.

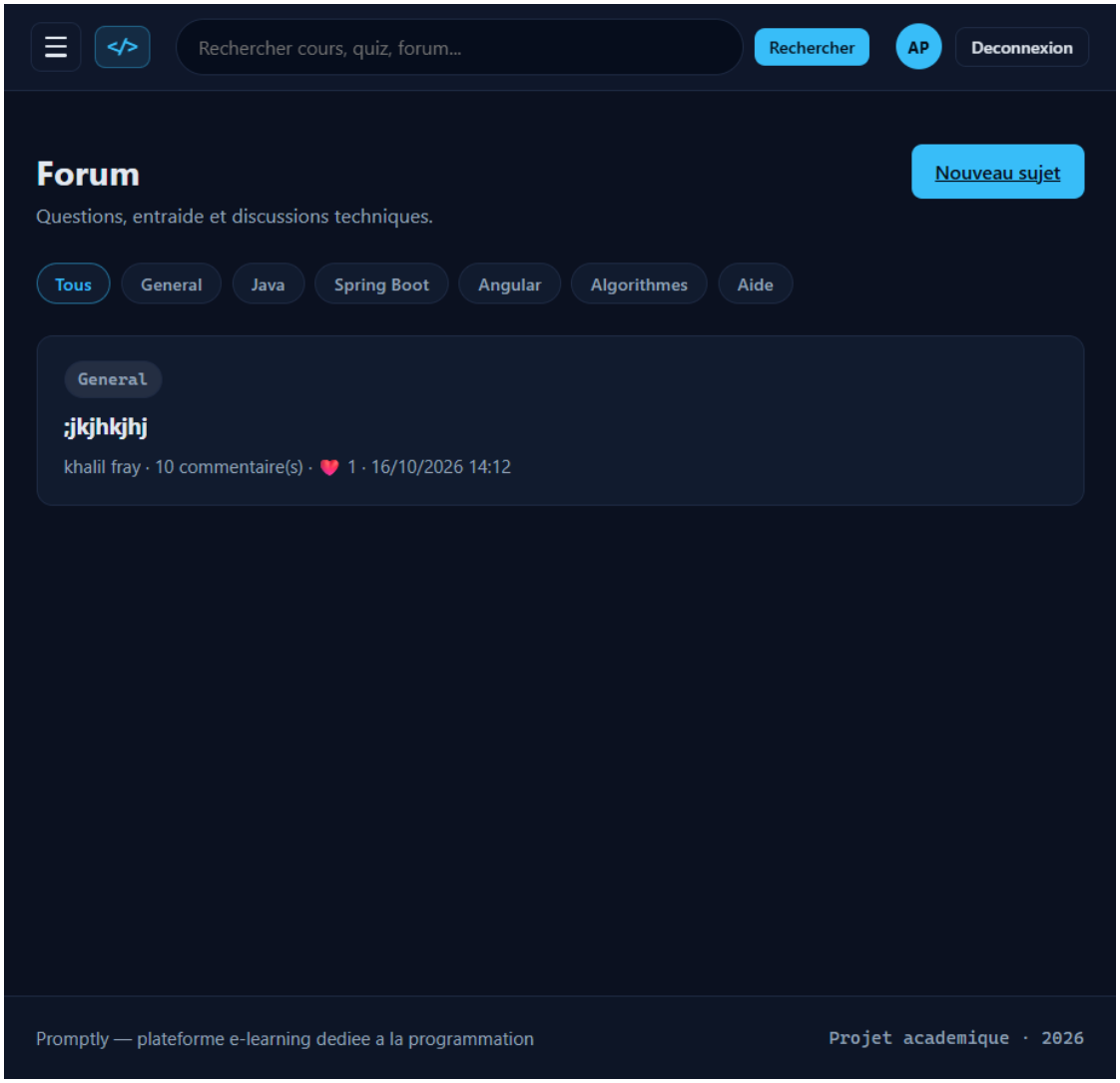


FIGURE 3.16 – Capture Promptly — forum (sujets, categories, reactions)

Prompt

add react to the forme like fb and img like fb post not just text

Resultat

Posts avec image ; reactions type Facebook (like, coeur, rire, ...) avec compteurs.

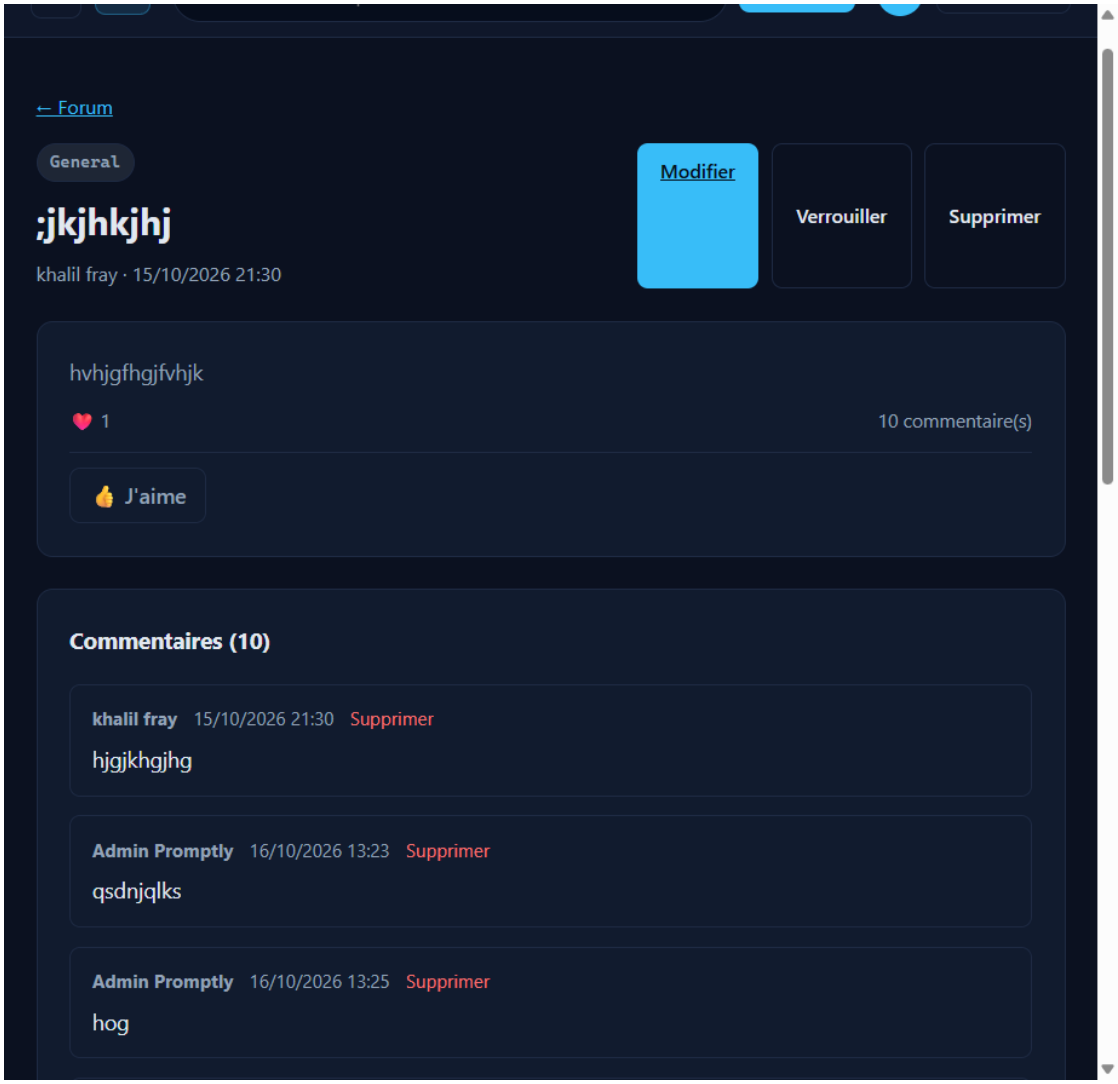


FIGURE 3.17 – Capture Promptly — detail sujet forum (reactions J'aime, commentaires)

Prompt

ok in the formome add api for badword bllocker ...make it with free one

Resultat (apres choix API a token)

Moderation via **Sightengine** (api_user + api_secret). Insultes remplacees par *** (au lieu d'un rejet).

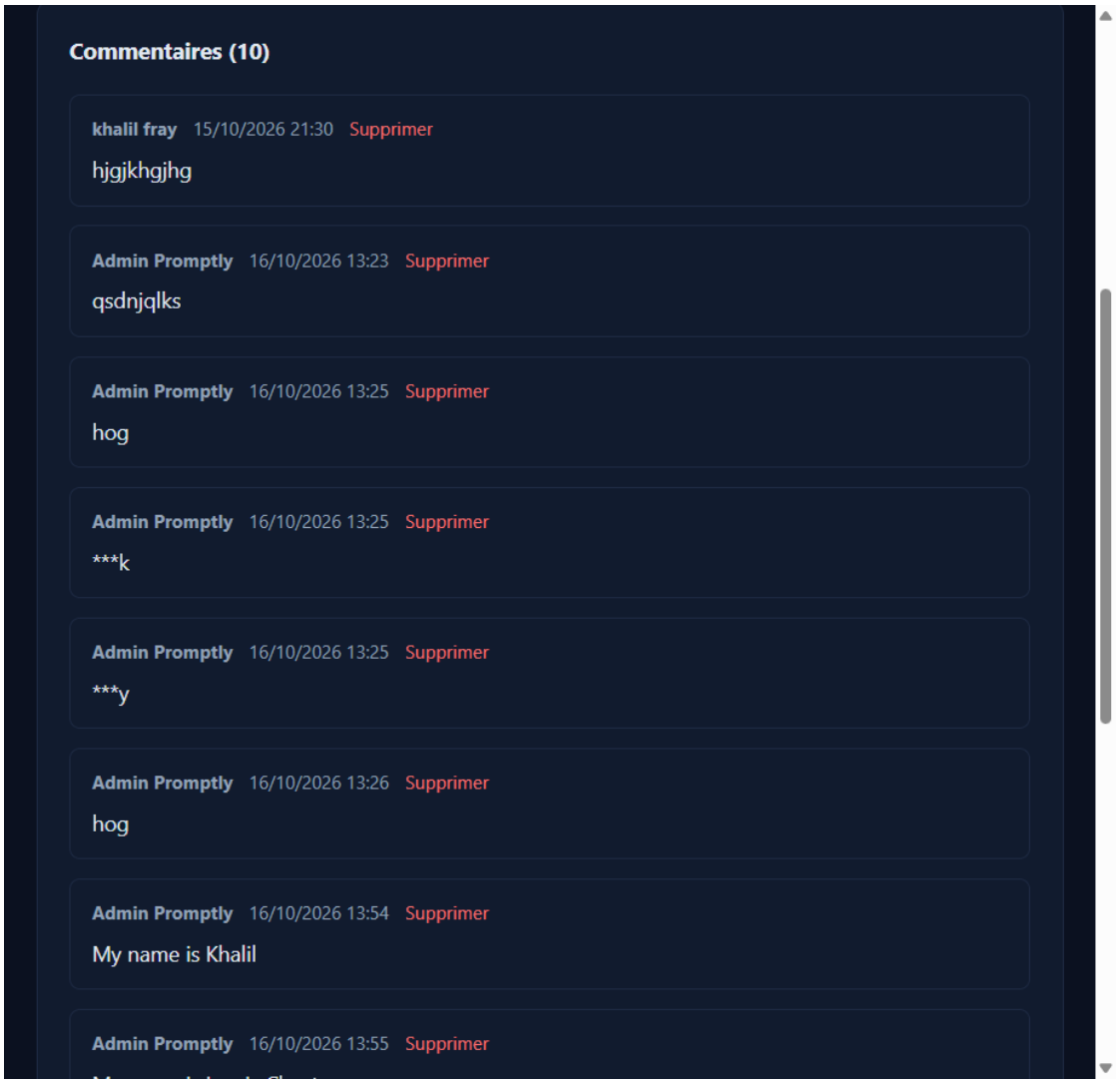


FIGURE 3.18 – Capture Promptly — commentaires avec censure *** (Sightengine)

Prompt

i need just when i type same thing wrong will auto correct when i post i need api with token make this and free

Probleme — APIs

GrammarBot annonce sa fermeture ; LanguageTool Premium refuse. **Correction prompt :** don't do that the professor told us make api do the auto correct → integration **TextGears** (cle API gratuite).

Resultat

A la publication d'un post/commentaire : correction orthographe TextGears (FR/EN configurable), puis censure Sightengine ***. Champ de reponse : « Votre reponse (insultes → ***) ».

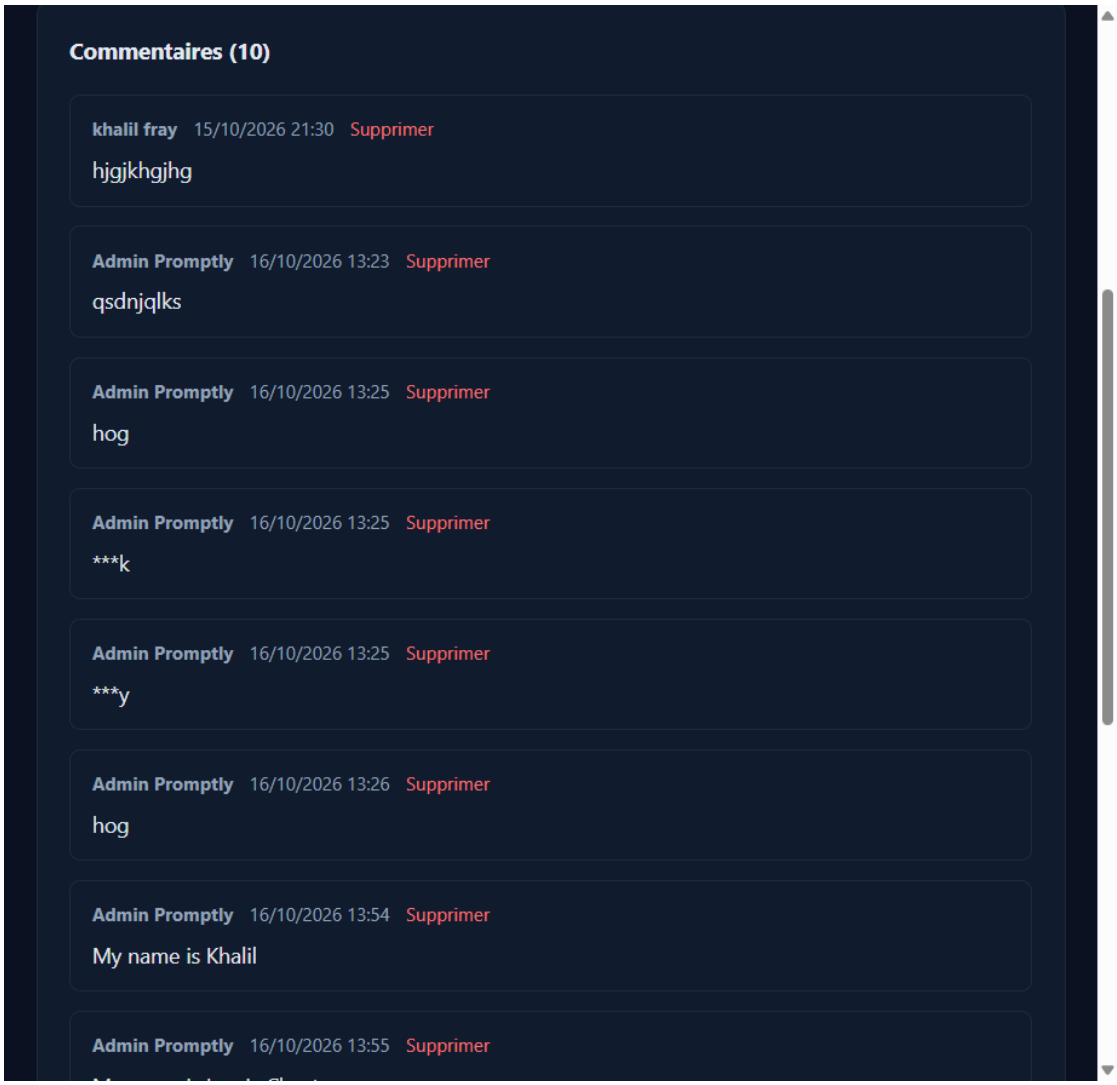


FIGURE 3.19 – Capture Promptly — moderation forum (TextGears + Sightengine ***)

3.3.6 S5 — Reclamations

Prompt

good make the next one

Resultat

Module reclamations : depot etudiant, suivi de statut ; admin repond et change statut (`OUVERTE` → `EN_COURS` → `RESOLUE`/`REJETEE`).

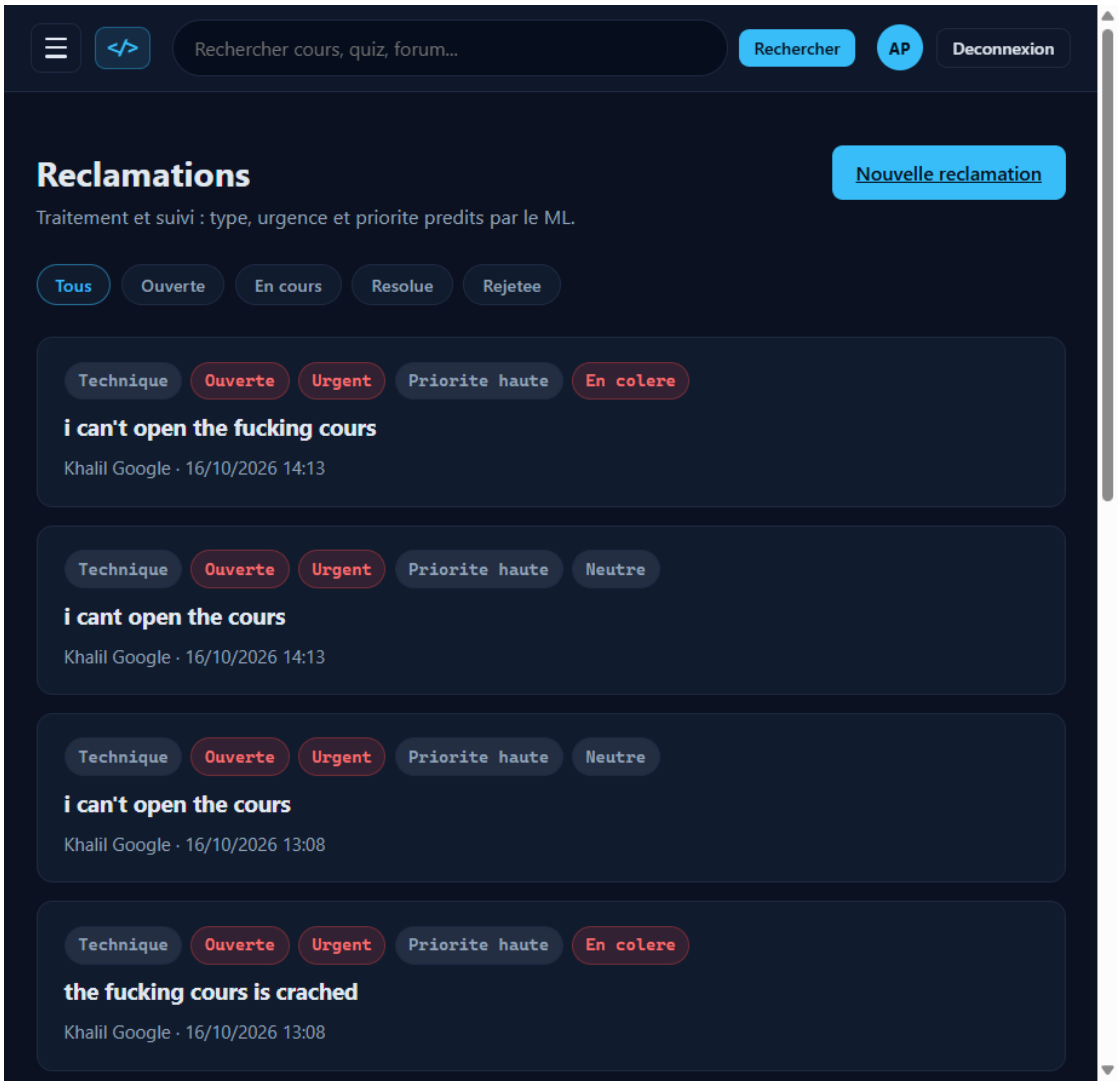


FIGURE 3.20 – Capture Promptly — liste des reclamations (filtres de statut)

Prompt

in the reclamation make other machine learing when same one make reclamtion the ml can classified urgence not urgence wand give each reclamtion preority and and the type the reclamtion technique disponibility exetera the admine can see this the user how make reclamtion he just make the reclamation

Resultat

Service `ml-complaints` (port 8091) : type, urgence, priorite. Visible admin seulement ; l'etudiant ne remplit que objet + description.

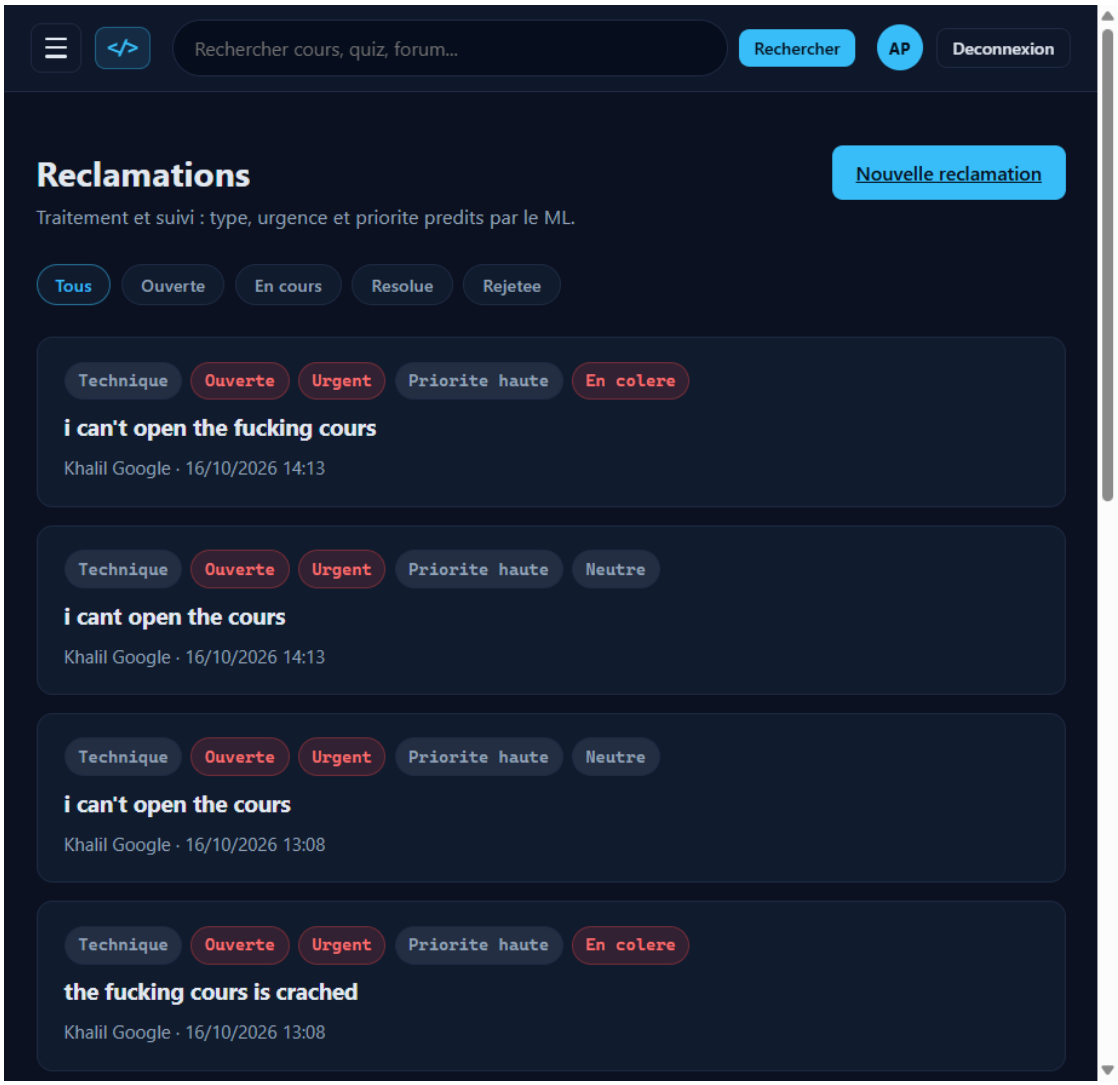


FIGURE 3.21 – Capture Promptly — tags ML type / urgence / priorite / emotion sur la liste

Correction classification

. . .when i type same urgence in the objet and the description u class it not urgence . . .don't make them all urgence make it about the object and description

Resultat

Modele reentraine : classification basee sur le texte objet+description (ex. quiz/deadline → technique / urgent ; suggestion calme → non urgent).

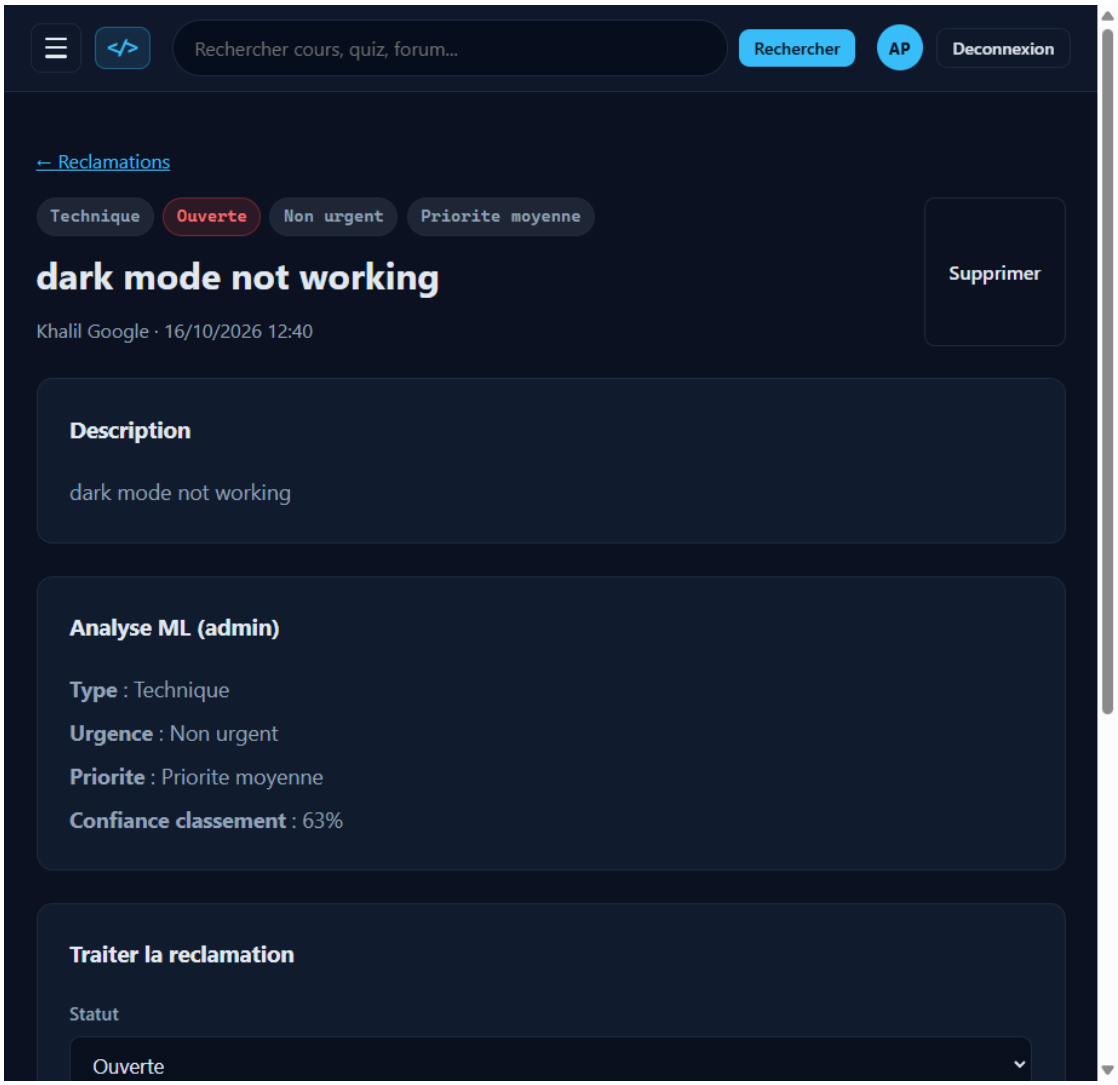


FIGURE 3.22 – Capture Promptly — detail avec classification ML (objet+description)

Prompt

ok now add emotion detaction in the reclation from the way he write the ia will told the admine is this persone angry or happy ..... use free api

Resultat

Emotion (En colere / Content / Triste / ...) via Hugging Face Inference + fallback lexique local, dans le backend Spring. Admin seulement. Panneau « Analyse ML (admin) » : type, urgence, priorite, confiance.

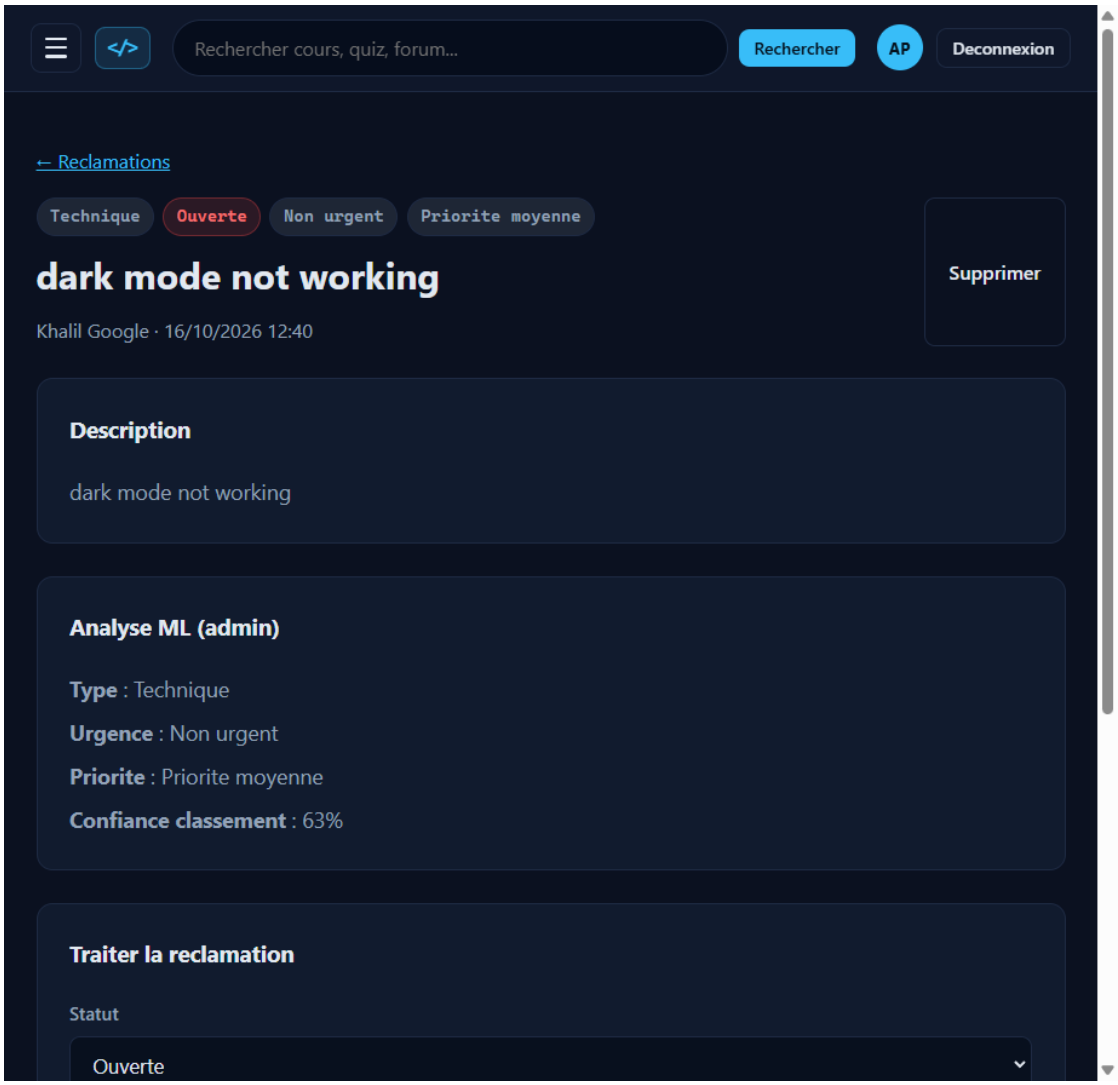


FIGURE 3.23 – Capture Promptly — detail reclamation avec Analyse ML (admin)

Probleme puis correction

Emotion reste « Neutre » malgre un texte agressif.
Causes : token HF ecrase / lexique sans mots vulgaires EN.
Correction : token local prioritaire + lexique enrichi → detection « En colere ».

3.3.7 S6 — Layout (navbar / sidebar)

Prompt

add side bar and make the nave bar have the login information and the name of the website and searsh bar

Resultat

Sidebar (navigation) + top bar (Promptly, recherche, infos login / logout).

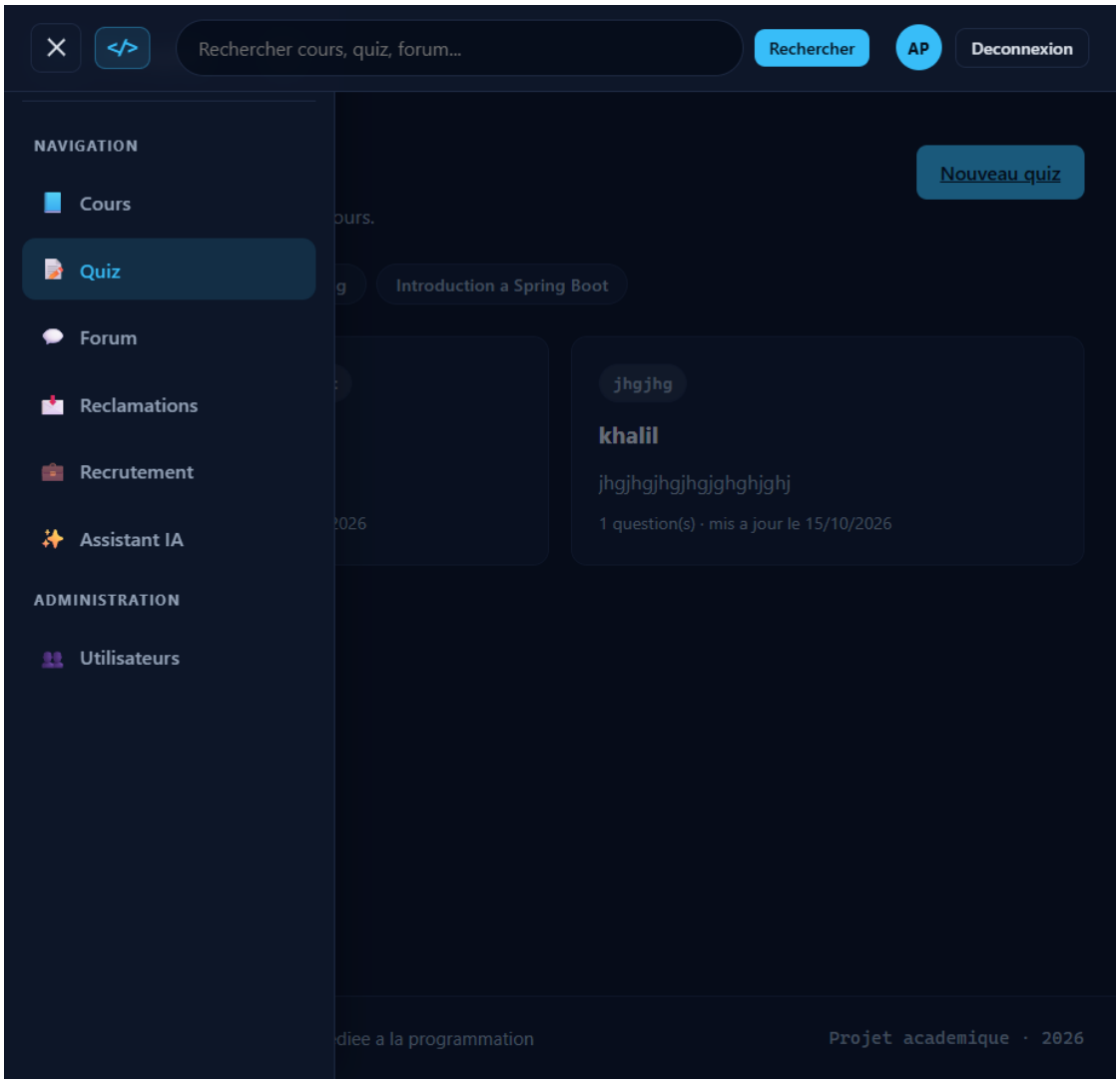


FIGURE 3.24 – Capture Promptly — layout sidebar + navbar + recherche

Prompt

make the side bar with toggel open and close when i click it

Probleme

when i click the side bar no thing is happen — host/overlay bloquait les clics.

Resultat apres correction

Bouton menu ouvre/ferme la sidebar ; backdrop mobile ; hard refresh.

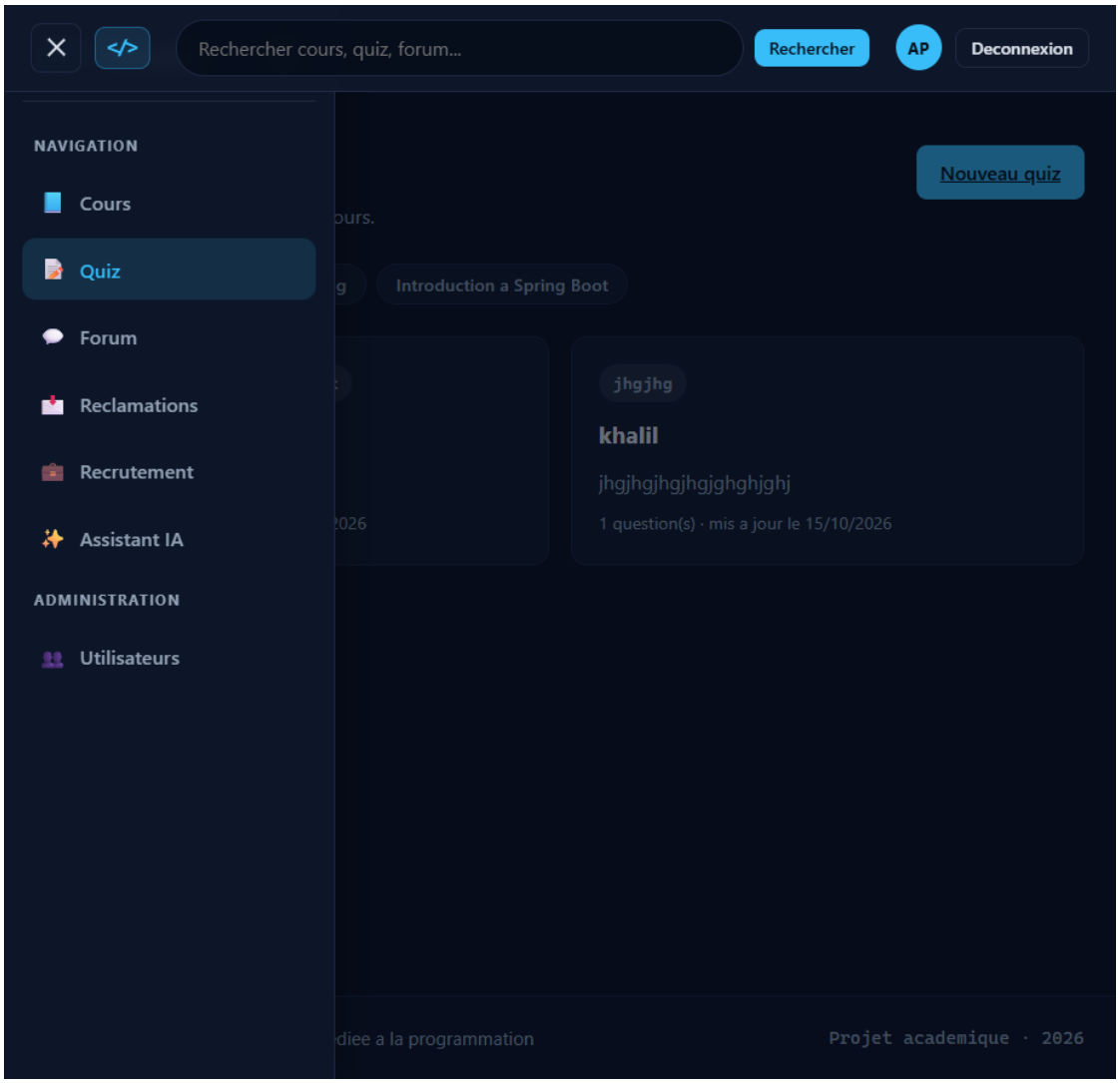


FIGURE 3.25 – Capture Promptly — sidebar ouverte + navbar (recherche, profil)

3.3.8 S7 — IA pedagogique (Ollama : resume, flashcards)

Prompt

i download ollama in the pc so make the resumer with ollama

Resultat

Bouton « Resumer automatiquement » sur cours/chapitre PDF : PDFBox → Ollama (`llama3.2:3b`) → resume.

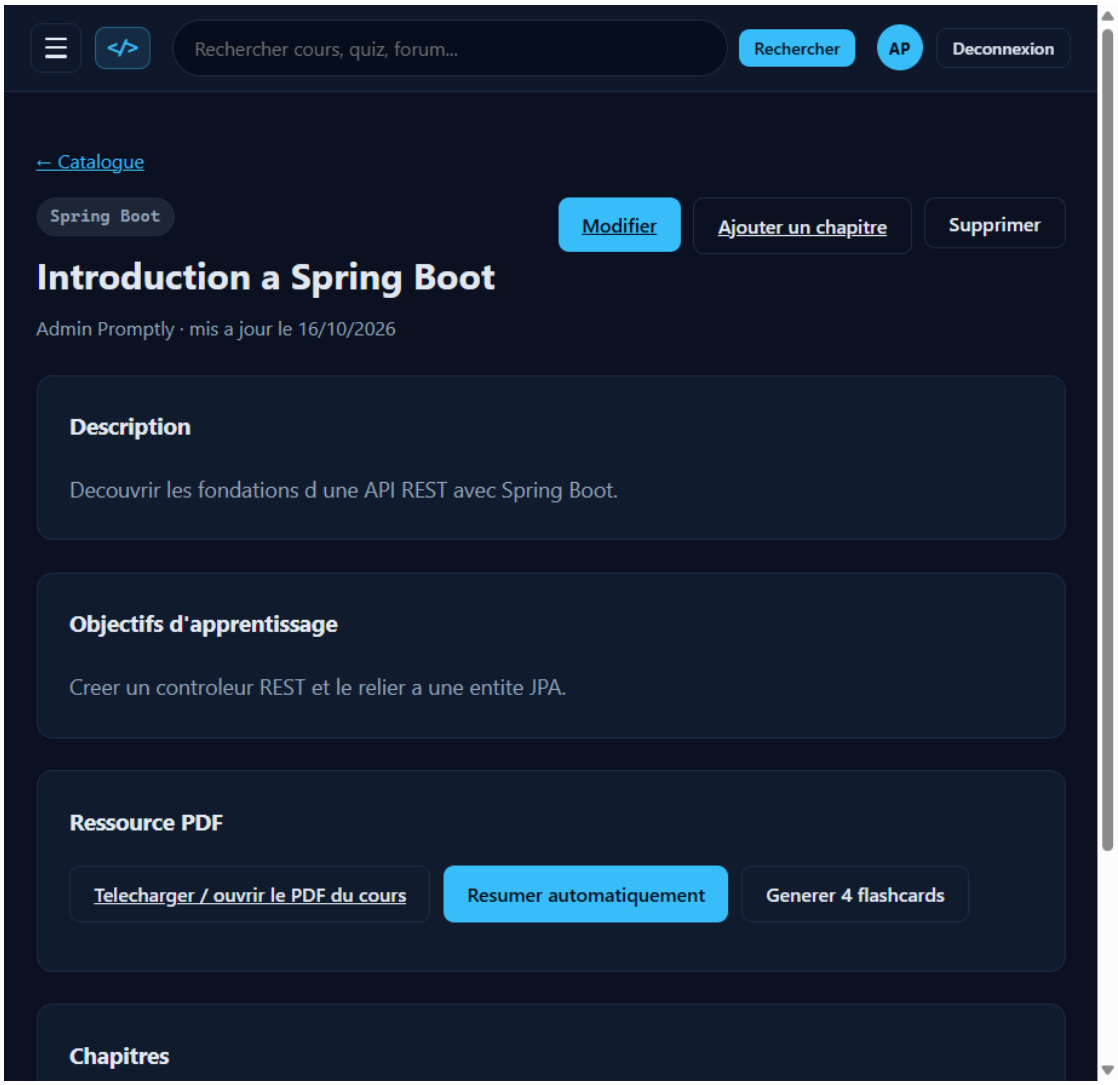


FIGURE 3.26 – Capture Promptly — bouton Resumer automatiquement (Ollama)

Prompt

and give like flush card of the cours read the pdf then give me 4 flash card . . .when i click them the card turn and i read the tips

Resultat

« Generer 4 flashcards » : 4 cartes recto/verso (titre / tip) depuis le PDF. Boutons visibles sur la fiche cours (avec le PDF).

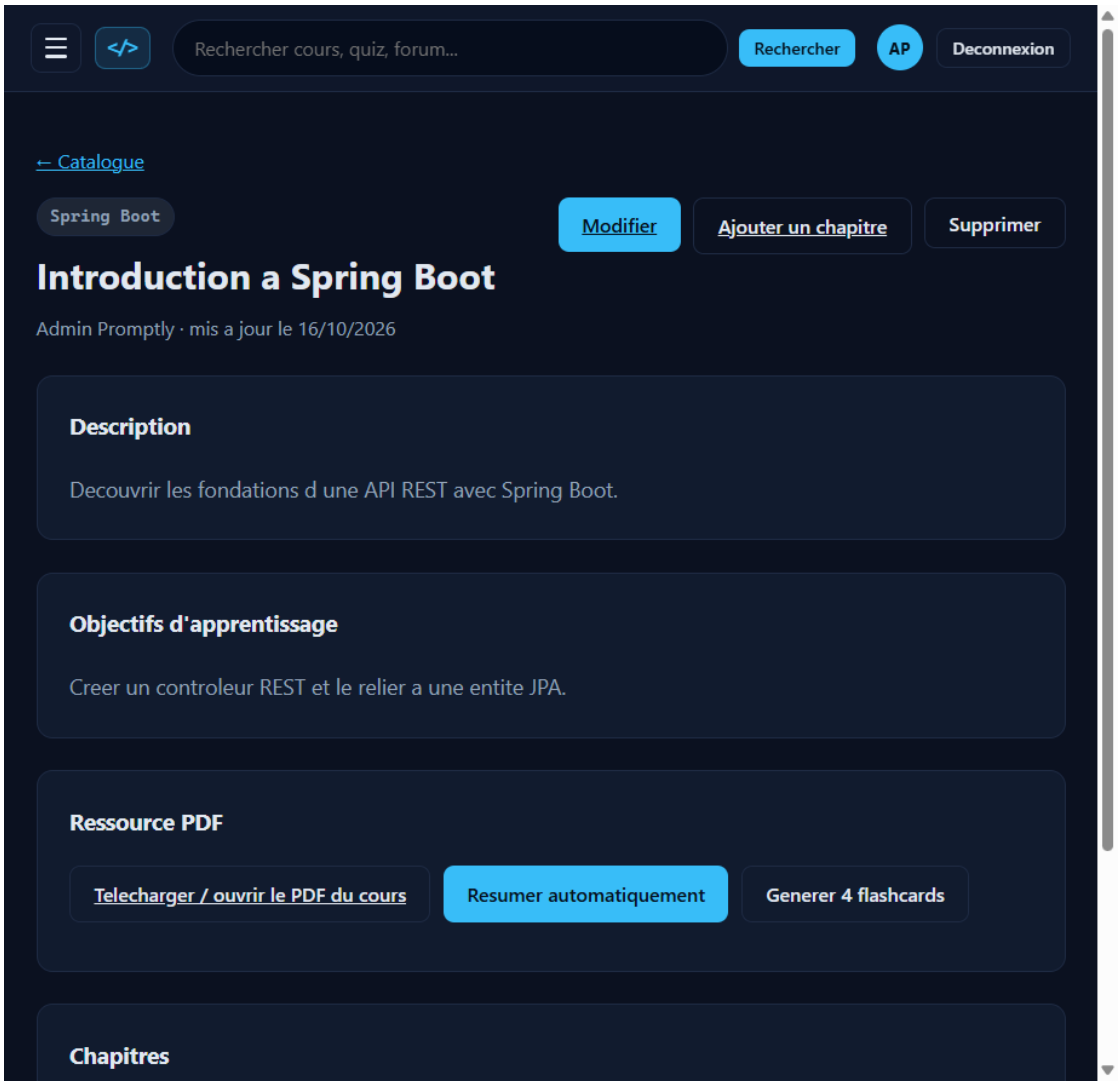


FIGURE 3.27 – Capture Promptly — Generer 4 flashcards (Ollama)

Probleme

Impossible d'interpreter les flashcards (400) ; cartes invisibles (CSS 3D). **Corrections :** parsing JSON Ollama renforce ; CSS flip simplifie.

3.3.9 S8 — Assistant IA (chatbot)

Prompt

assistance ia make chatbot general for the aplication make it like llm

Resultat

Page `/ia` : chat general via `POST /api/ai/chat` (Ollama), historique de conversation, suggestions. Necessite Ollama + modele local.

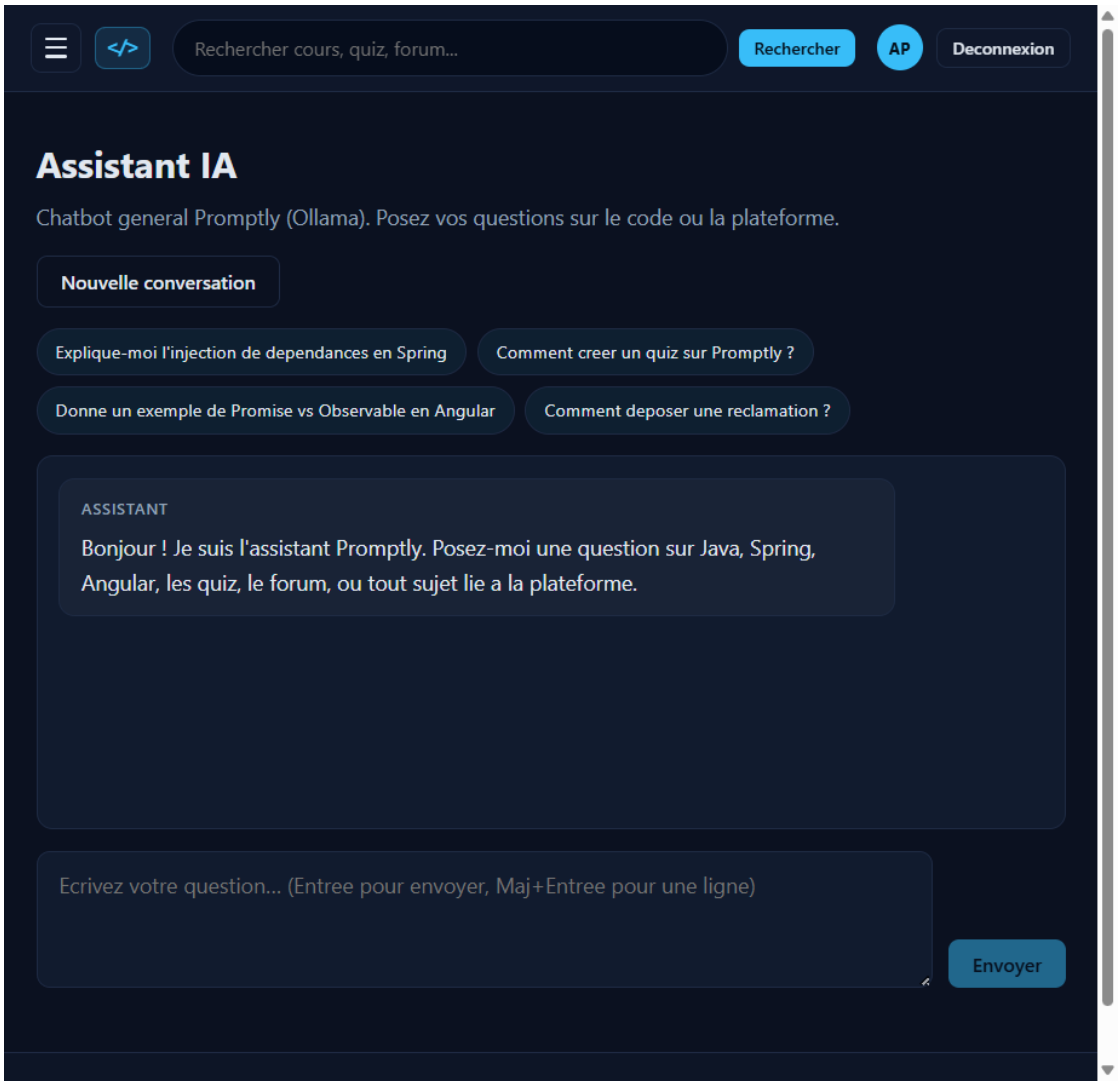


FIGURE 3.28 – Capture Promptly — Assistant IA (chatbot Ollama)

3.3.10 S9 — Recrutement + matching CV

Prompt

ok we need to add other gestion like recretment the admine make recretment then make the employee see the offre and add his cv then the admine see how aplly the offre and see his cv

Resultat

Module `/recrutement` : admin cree offres ; candidats postulent avec CV PDF ; admin voit candidatures + CV + statuts.

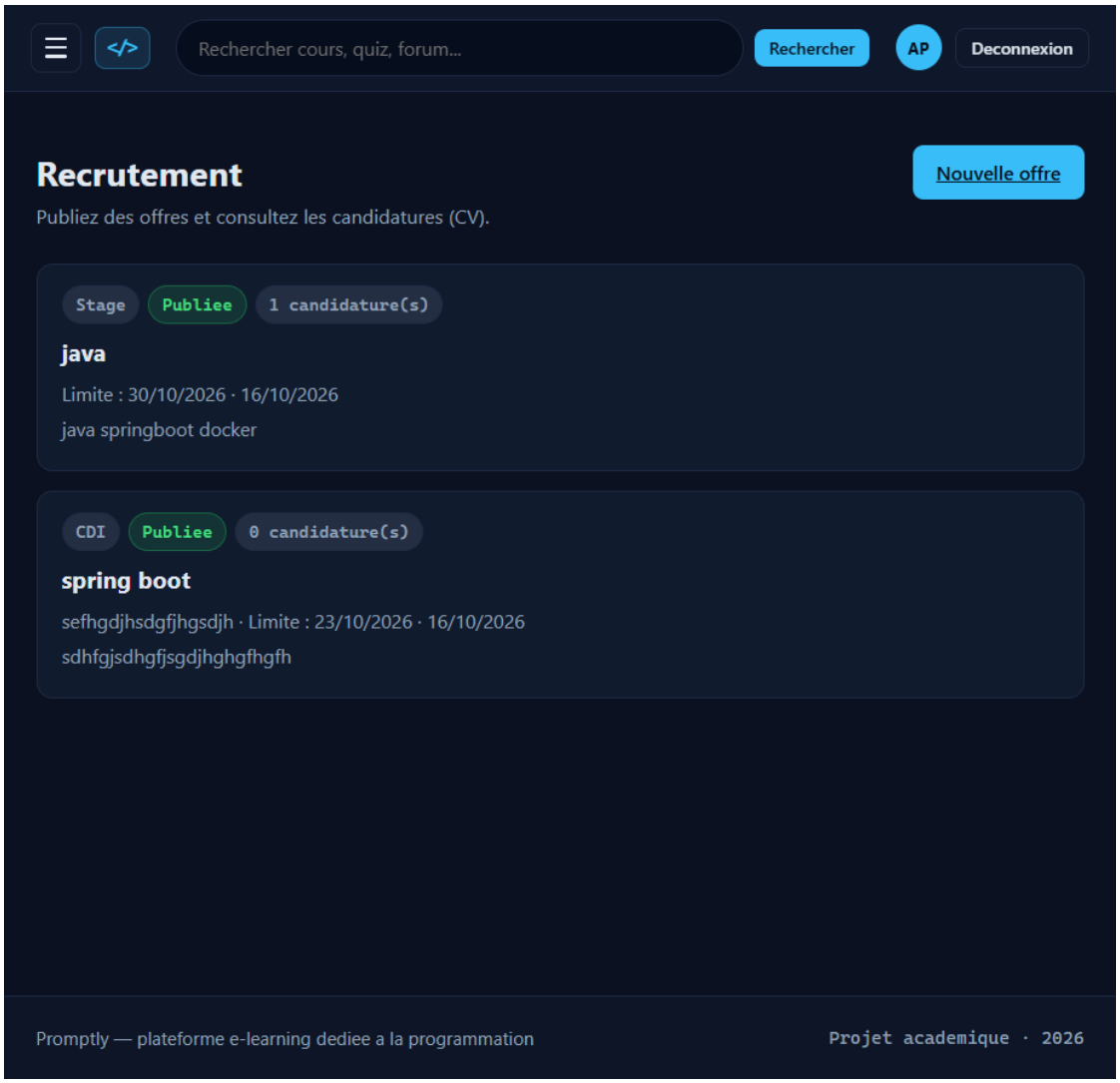


FIGURE 3.29 – Capture Promptly — liste des offres de recrutement

Prompt

good now add part of skills and experience in the offre form then when the employee upload his cv make matching cv with the skkils in the offre and the skills in the cv use api with token to make this and be sure the api free

Resultat (version initiale)

Skills + experience sur l'offre ; matching via Hugging Face (`bart-large-mnli`) → score %, competences matched/missing.

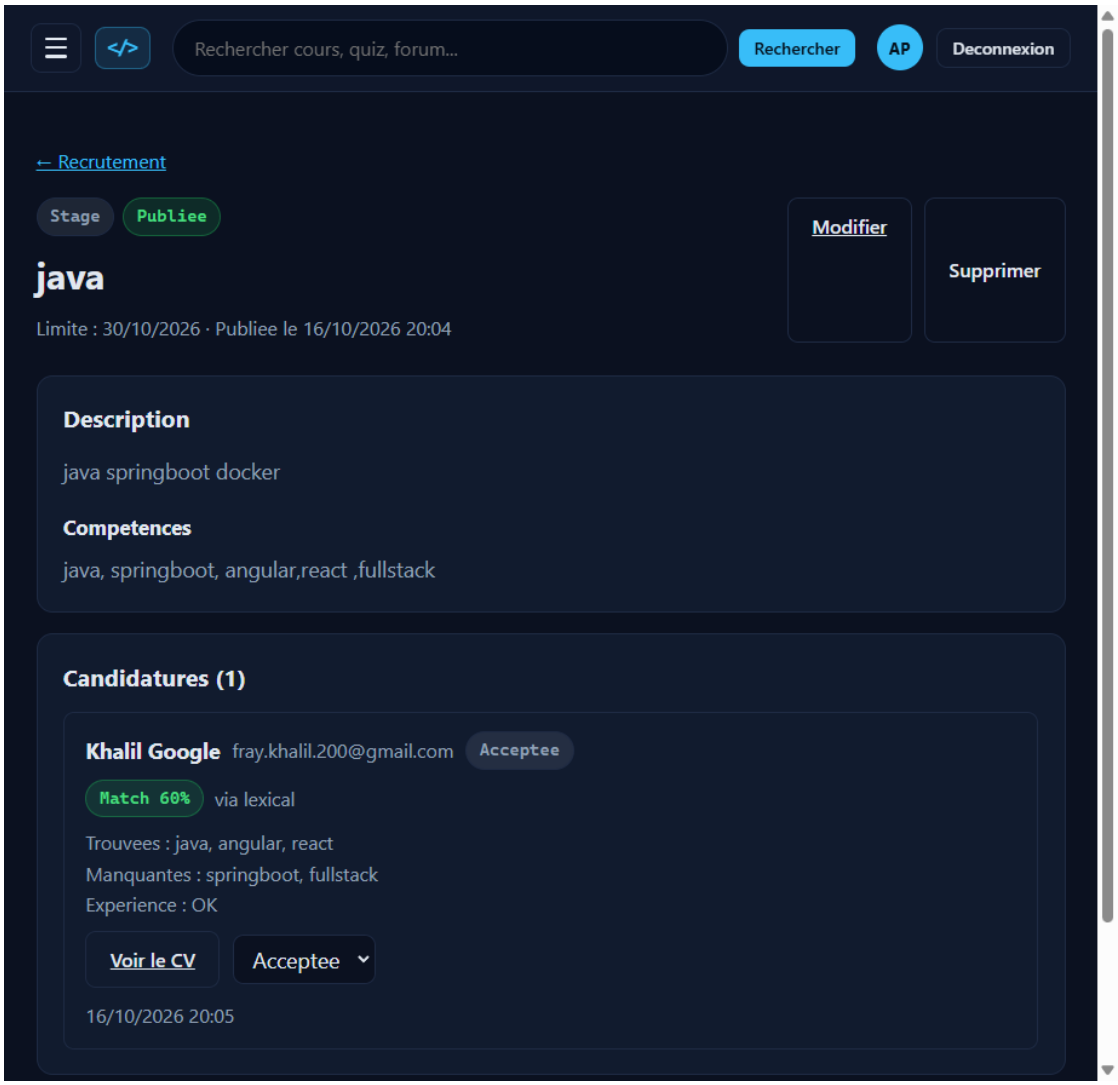


FIGURE 3.30 – Capture Promptly — offre avec competences et score de matching CV

Correction — changement d'API

can u relpalce Hugging Face just in this gestion with other pai to make matching cv ?

Resultat

Matching CV bascule sur **Google Gemini** (cle gratuite AI Studio). HF reste pour l'emotion des reclamations. Fallback mots-cles si API down. Admin voit le score Match %, competences trouvees / manquantes.

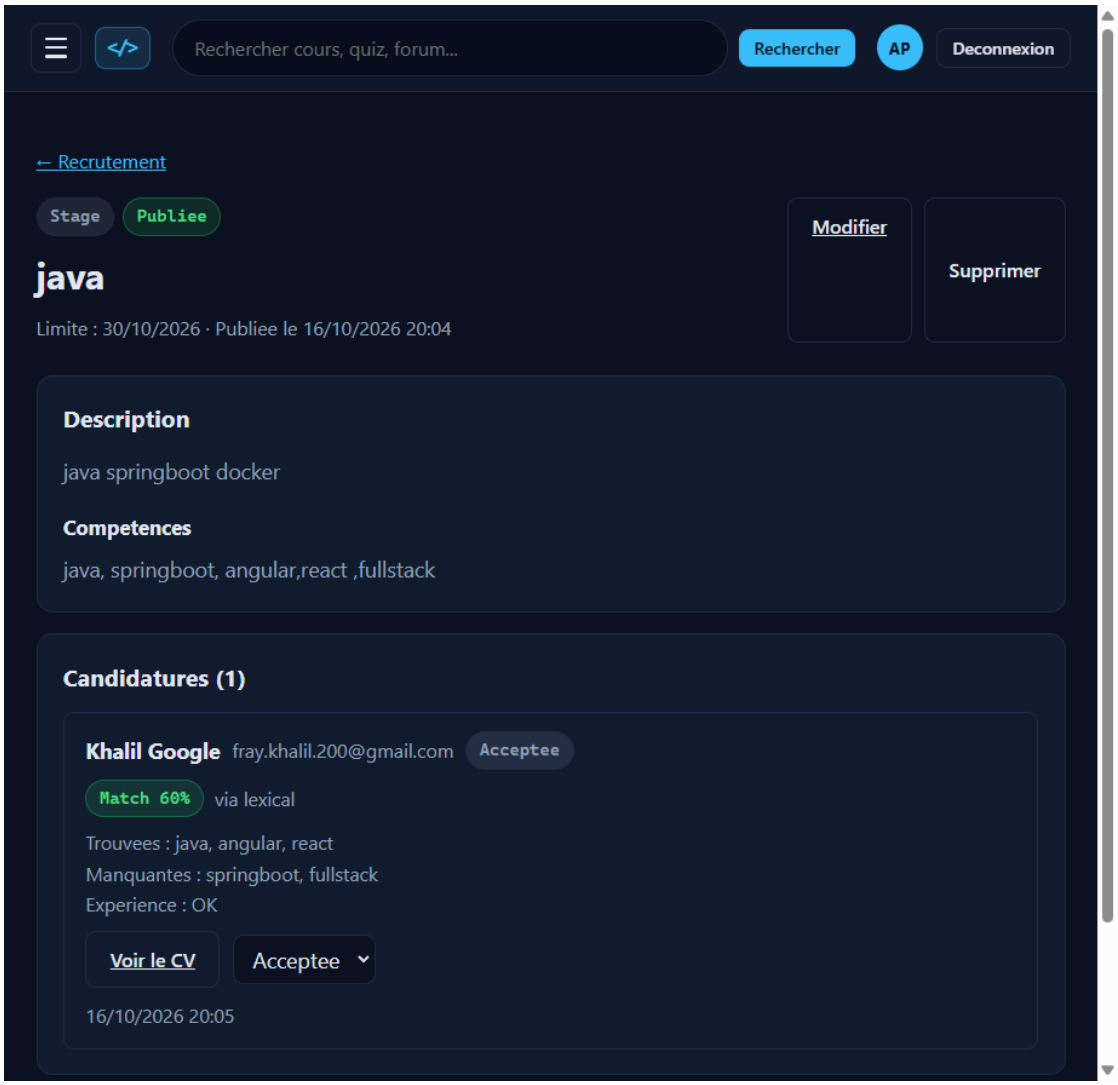


FIGURE 3.31 – Capture Promptly — offre avec matching CV (score + competences)

3.4 Synthese des sessions

Session	Gestion	Resultat principal
S0	Fondation	Spring Boot + Angular + MySQL + health
S1	Auth (+ Google)	JWT, roles, Sign-In Google
S2	Cours / chapitres	CRUD, image, PDF
S3	Quiz + ML	QCM, Ollama, timers, GUESS/KNOW
S4	Forum	Posts, reactions, Sightengine, TextGears
S5	Reclamations	ML type/urgence + emotion HF
S6	Layout	Sidebar / navbar / recherche
S7	Resume / flashcards	Ollama + PDFBox
S8	Assistant IA	Chatbot Ollama
S9	Recrutement	Offres, CV, matching Gemini

TABLE 3.2 – Recapitulatif des gestions documentees depuis l'historique Cursor

Chaque resultat ci-dessus est illustre par une **capture de la plateforme Promptly** (`figures/app/`). Des captures Cursor (prompt dans le chat Agent) restent optionnelles dans `figures/cursor/`.

Chapitre 4

IA integree a l'application

Ce chapitre decrit **comment l'intelligence artificielle est utilisee dans le produit Promptly** (cote application), et non comment le code a ete genere avec Cursor / Claude / Grok. Deux familles d'outils coexistent :

— **Ollama** : modele de langage local (LLM) ;

— **APIs externes** et **microservices ML** : services specialises (moderation, emotion, matching, prediction).

Les cles API sont stockees dans `application-local.properties` (fichier gitignore), jamais dans le depot Git. Documentation detaillee : `docs/apis-et-tokens.md`.

4.1 Vue d'ensemble des fonctionnalites

Fonctionnalite	Technologie	Role
Resume PDF	Ollama + PDFBox	Resume automatique d'un cours / chapitre
Flashcards	Ollama + PDFBox	4 cartes pedagogiques depuis le PDF
Generation de quiz	Ollama	10 QCM a partir du titre / cours / PDF
Assistant IA	Ollama	Chatbot general dans l'application
Emotion reclamations	Hugging Face	Ton (colere, neutre, ...) pour l'admin
Censure forum	Sightengine	Insultes remplacees par ***
Auto-correct forum	TextGears	Orthographe / grammaire a la publication
Matching CV / offre	Google Gemini	Score competences + experience
GUESS / KNOW quiz	ml-integrity (:8090)	Reponse au hasard vs connue
Type / urgence	ml-complaints (:8091)	Labels admin sur reclamations

Table 4.1 – Fonctionnalites d'IA integrees a Promptly

4.2 Ollama — LLM local

4.2.1 Pourquoi Ollama

Ollama permet d'executer un modele de langage **sur la machine de developpement** (`http://localhost:11434`), sans cout API pour les fonctions pedagogiques centrales. Les donnees des PDF et des conversations restent locales. Modele utilise : `llama3.2:3b` (ou equivalent installe via `ollama pull`).

Extraction de texte des PDF : bibliotheque Java **Apache PDFBox** dans le backend Spring Boot, avant envoi du texte a Ollama.

4.2.2 Resume automatique de PDF

Sur la fiche d'un **cours** ou d'un **chapitre** ayant un PDF :

1. l'utilisateur clique sur « Resumer automatiquement » ;
2. le backend extrait le texte du PDF (tronque si trop long) ;
3. Ollama produit un resume en francais ;
4. le resume s'affiche sous la section PDF.

Endpoints typiques : `POST /api/ai/courses/{id}/summarize-pdf` et equivalent chapitre. Limitation : un PDF scanne (image seule, sans texte selectable) ne peut pas etre resume.

4.2.3 Flashcards

Meme flux PDF → texte → Ollama, avec une consigne demandant **4 flashcards** (recto = titre / question, verso = tip). Bouton « Generer 4 flashcards » ; l'utilisateur retourne les cartes dans l'interface Angular. Le backend parse la reponse JSON d'Ollama (avec fallbacks si le format est imparfait).

4.2.4 Generation de quiz (10 questions)

Dans le formulaire « Nouveau quiz », apres titre, description et choix du cours (chapitre optionnel) :

1. clic sur « Generer 10 questions avec IA » ;
2. Ollama recoit titre, description et contenu du cours / chapitre / PDF ;
3. 10 QCM sont proposes ; l'enseignant peut les modifier avant creation.

La generation peut prendre 30 a 90 secondes selon la machine.

4.2.5 Assistant IA (chatbot)

Page `/ia` : chatbot general de la plateforme.

— Backend : `POST /api/ai/chat` avec historique de conversation ;

— Frontend : bulles de chat, suggestions de questions, envoi par Entree ;

— Pre-requis : Ollama demarre et modele telecharge.

L'assistant repond a des questions sur Java, Spring, Angular, le fonctionnement de Promptly, etc. Ce n'est pas un remplacement du professeur : c'est une aide contextuelle integree a l'application.

4.3 APIs externes

4.3.1 Hugging Face — emotion des reclamations

Usage. Analyser le **ton** d'une reclamation (En colere, Content, Triste, Inquiet, Surpris, Neutre) a partir du texte objet + description.

Qui voit le resultat. Uniquement l'**administrateur** (panneau Analyse ML). L'etudiant qui depose la reclamation ne voit pas l'emotion.

Technique. Appel Inference API Hugging Face depuis le backend Spring (`ComplaintEmotionService`). Token : `promptly.emotion.hf-token`. Si l'API est indisponible, un **lexique local** FR/EN sert de secours.

4.3.2 Sightengine — moderation du forum

Usage. Detecter les insultes / gros mots dans les **posts** et **commentaires** du forum, puis les remplacir par `***` (plutot que de rejeter le message).

Technique. API Sightengine (plan gratuit) avec `api-user` et `api-secret` dans `application-local.proper` Service Java : `ProfanityFilterService`. Sans cles : liste locale de mots.

4.3.3 TextGears — auto-correction du forum

Usage. Corriger l'orthographe et la grammaire **a la publication** d'un post ou d'un commentaire (avant la censure Sightengine).

Technique. API TextGears avec cle `promptly.spellcheck.api-key` ; langue configurable (`fr-FR` ou `en-US`). Service : `SpellCorrectService`. Le professeur exigeait une API avec token : TextGears a ete retenu apres essais d'autres fournisseurs (LanguageTool Premium, GrammarBot ferme, etc.).

4.3.4 Google Gemini — matching CV / offre

Usage. Dans le module **Recrutement**, apres depot d'un CV PDF sur une offre contenant des **competences** (et eventuellement une experience) :

1. extraction du texte du CV (PDFBox) ;
2. Gemini compare le CV aux competences de l'offre ;
3. le backend stocke un **score Match %**, les competences trouvees / manquantes, et un indicateur d'experience.

Visible par l'**admin** sur la fiche de candidature. Cle gratuite AI Studio : `promptly.cv-match.api-key`. Sans cle : matching lexical (mots-cles) en secours. Hugging Face n'est **plus** utilise pour cette fonction (uniquement pour l'emotion).

4.4 Microservices ML locaux (Python)

4.4.1 ml-integrity (port 8090) — GUESS / KNOW

Lorsqu'un etudiant passe un quiz (une question a la fois, avec minuteurs), le service Python `ml-integrity` (scikit-learn, RandomForest) predit si la reponse est :

— `GUESS` : reponse au hasard ;

— `KNOW` : reponse probablement connue.

Features utilisees : temps passe, changements de reponse, etc. **Seul l'enseignant** voit ces labels (Detail + analyse ML). Une reponse fausse est forcee en `GUESS`. Si le service est arrete, Spring applique une heuristique locale.

4.4.2 ml-complaints (port 8091) — type, urgence, priorite

A chaque nouvelle reclamation, le service `ml-complaints` predit :

— **type** (technique, pedagogique, compte, ...) ;

— **urgence** (urgent / non urgent) ;

— **priorite** (haute / moyenne / ...).

L'etudiant ne remplit que l'objet et la description ; les labels sont reserves a l'admin. La classification s'appuie sur le **texte** (pas uniquement sur des mots-cles fixes). Emotion = service HF separe (voir ci-dessus), pas ce microservice.

4.5 Synthese

— **Ollama** couvre le coeur pedagogique : resume, flashcards, generation de quiz, assistant chatbot — gratuit et local.

— **APIs externes** couvrent des besoins specialises : emotion (HF), moderation (Sightengine), orthographe (TextGears), matching CV (Gemini).

— **ML Python** couvre des predictions metier : integrite des reponses au quiz, tri des reclamations.

— En cas d'indisponibilite d'une API ou d'un service ML, des **fallbacks** locaux limitent la panne de l'application.

Chapitre 5

Conclusion

5.1 Bilan

L'IA a ete utilisee sur **deux faces** :

— **Developpement** : ChatGPT pour amorcer le cahier des prompts ; editeur **Cursor** avec modeles **Claude** et **Grok** pour implementer et corriger le code de facon iterative ;

— **Produit** : Ollama et APIs (HF, Sightengine, TextGears, Gemini) plus des modeles ML locaux pour des besoins metier precis.

5.2 Limites

— les modeles peuvent proposer du code incorrect : verification et prompts de correction sont indispensables ;

— les APIs gratuites ont des quotas et peuvent etre indisponibles ;

— les captures et prompts doivent etre reels (pas reconstitues).

5.3 Perspectives

Enrichir le journal de prompts avec toutes les sessions (en indiquant Claude ou Grok), completer les figures, et ajouter d'autres captures des fonctionnalites IA en situation reelle d'utilisation.